Module 2

1. Transposition Ciphers

- Introduction to Transposition Ciphers
- Keyless Transposition
- Keyed Transposition
- Examples / techniques

2. Introduction to Modern Symmetric-Key Ciphers

- Modern Block Ciphers
- Components of Modern Block Ciphers
- Two Classes of Product Ciphers
- Attacks for Block Ciphers

. Data Encryption Standard (DES)

- DES History
- DES Structure
- DES Analysis
- Multiple DES
- Security of DES

1. Transposition Ciphers

Introduction

- **Transposition** means **changing the position of letters**.
- The letters themselves are **not changed**.
- Only the **order of letters** is changed.
- Plaintext is rearranged to get ciphertext.

Keyless Transposition

- **No key** is used.
- Letters are arranged in a fixed pattern.
- The order of reading is changed.
- **Examples:**
  - Rail Fence Cipher
  - Row-by-row and column-by-column method

Keyed Transposition

- A **key** is used.
- The key decides the **order of columns**.
- Different keys can produce different ciphertexts.

- **Permutation Key**
- **Keyed Columnar Transposition**
- **Double Transposition**

Transposition Technique in Cryptography

- 1.Rail Fence Cipher

Transposition Technique

Rail Fence Cipher –Encryption

Steps:

1.Write the message in a zigzag pattern (rails or depth).

2.Read the letters row by row.

3. keyless cipher

4. The total number of letters in the message is equal to the total column.

. Note :

- If the text is not a multiple of **2**, add extra letters to the **plaintext**.
- This makes the plaintext length a **multiple of 2**.
- This step is used in the **Rail Fence Cipher**.
- Depth : means group of two letters

- **Plain Text:** `"HELLO"`
- **Length of Plain Text (ℓ):** `5`
- **Depth (Rails) (n):** `2`

Formula:

$$\frac{\text{Length of Text}}{2(\text{Depth} - 1)}$$

Applying Values:

$$\frac{5}{2 \times (2 - 1)}$$

$$\frac{5}{2}$$

Since **5 is not divisible by 2**, we need to add **1 extra letter** (padding) to make it **6**.

Note :

Since **5 is not divisible by 2**, we need to add **1 extra letter** (padding) to make it **6**.

Updated Plain Text: `"HELLOX"`

Now, the length ℓ = **6**, and

$$\frac{6}{2 \times (2-1)} = \frac{6}{2} = 3$$

Now, we can divide the text properly in rails.

Transposition Technique

Rail Fence Cipher –Encryption

Steps:

1. Write the message in a zigzag pattern (rails or depth).
2. Read the letters row by row.
3. keyless cipher
4. The total number of letters in the message is equal to the total column.

Question : encryption and decryption

- Message: **"HELLO WORLD“** **(A total of 10 letters, so 10 columns.)**
  depth : **2 (two row)**

Encryption steps

Step 1: Choose a Plaintext
- Select a plaintext.
- Letters should be **odd in number**.
- Should **not be a multiple of 2**.
- Example: **"RAILFENCE"**
- Total letters = **9**

Step 2: Use the Formula to Find Required Length
- Use the formula: **Length / 2(depth − 1)**
- Update the **new depth** if needed.

Step 3: Adjust the Text Length
- Add extra letters (like **Z**) if required.
- Make the length suitable for the formula.

Step 4: Fill the Table in Rail Fence Pattern
- Write letters in **zigzag (rail fence) style**.
- Follow the **depth rows**.

Step 5: Read Row-wise
- Read the letters **row by row** to get the result.

Example: step 1 : encrypt from PLAIN TEXT TO CIPHER TEXT

- Message: **"HELLO WORLD“** **(A total of 10 letters, so 10 columns.)**
  depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern

Columns

Rows

	1	2	3	4	5	6	7	8	9	10
1										
2										

Example: step 1 : encrypt from PLAIN TEXT TO CIPHER TEXT

Place the first letter "H" in the 1st row, 1st column.

- Message: **"HELLO WORLD"** **(A total of 10 letters, so 10 columns.)**
  depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern (Zaw pattern)

Columns

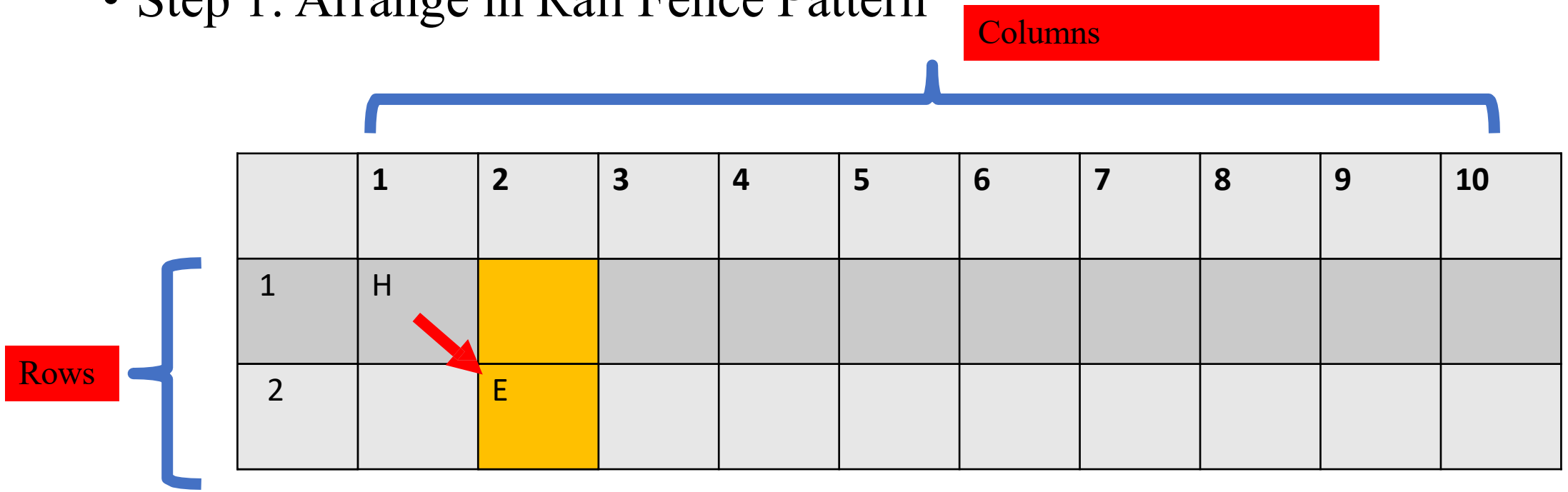
Rows

	1	2	3	4	5	6	7	8	9	10
1	H									
2										

Place **E** in **2nd row, 2nd column**.

Example: encrypt from PLAIN TEXT TO cipher text

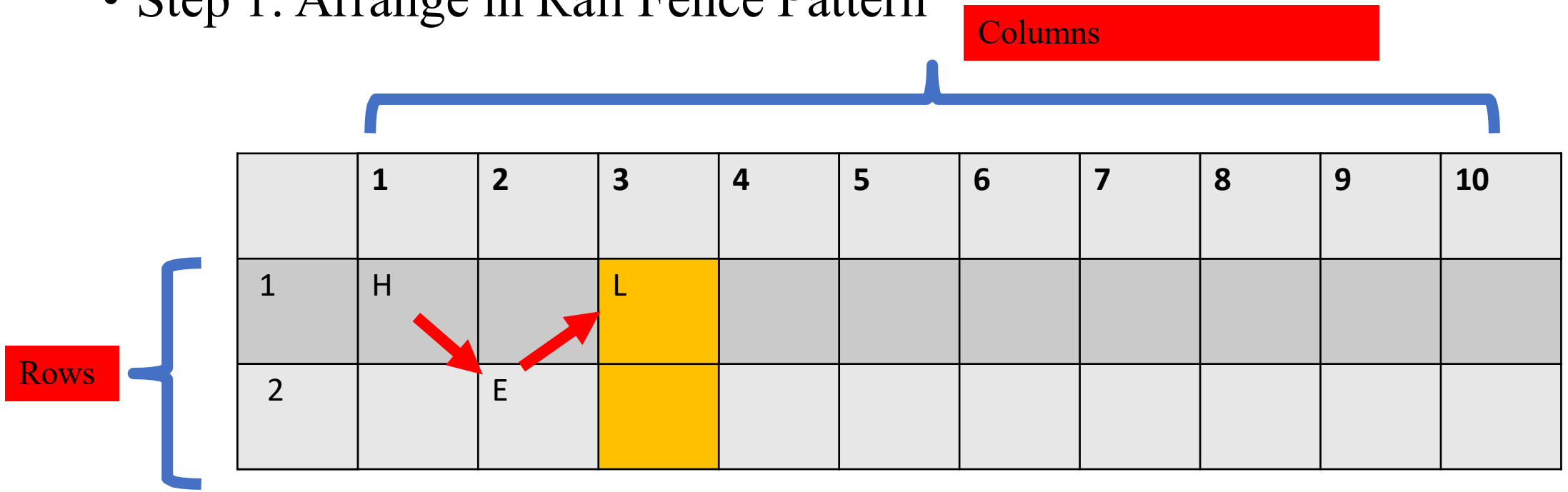
- Message: **"HELLO WORLD“ (A total of 10 letters, so 10 columns.)** depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern



Example: encrypt from PLAIN TEXT TO cipher text

Place L in 1st row, 3rd column.

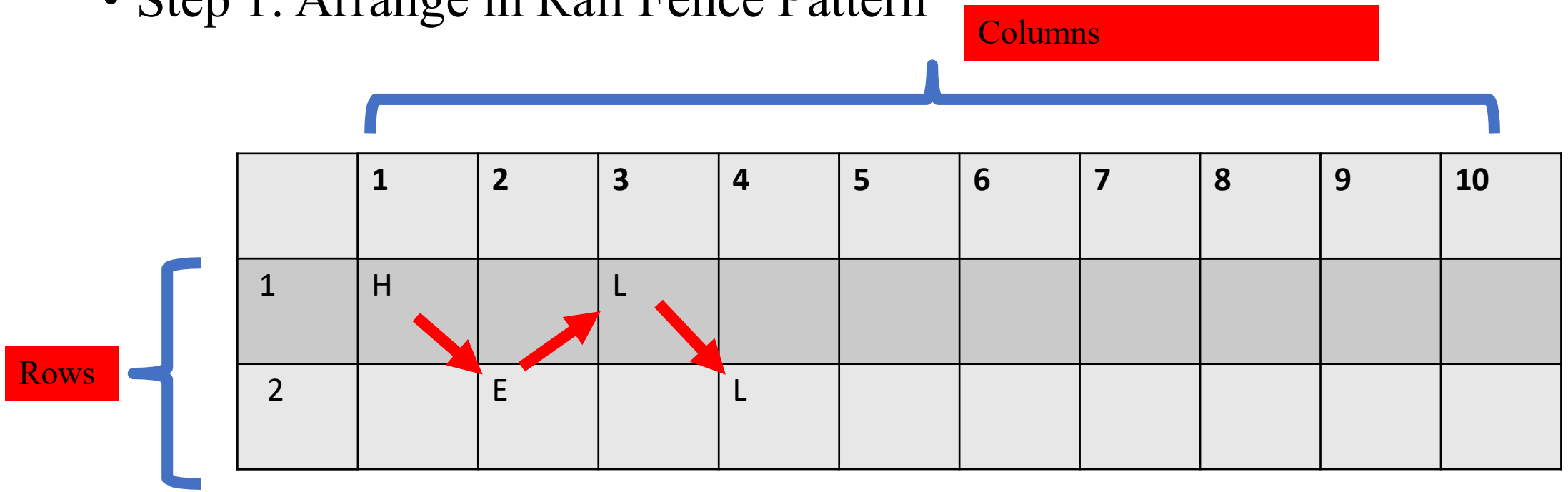
- Message: **"HELLO WORLD“ (A total of 10 letters, so 10 columns.)** depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern



Example: encrypt from PLAIN TEXT TO cipher text

Place **L** in **2nd row, 4th column**.

- Message: **"HELLO WORLD“ (A total of 10 letters, so 10 columns.)** depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern



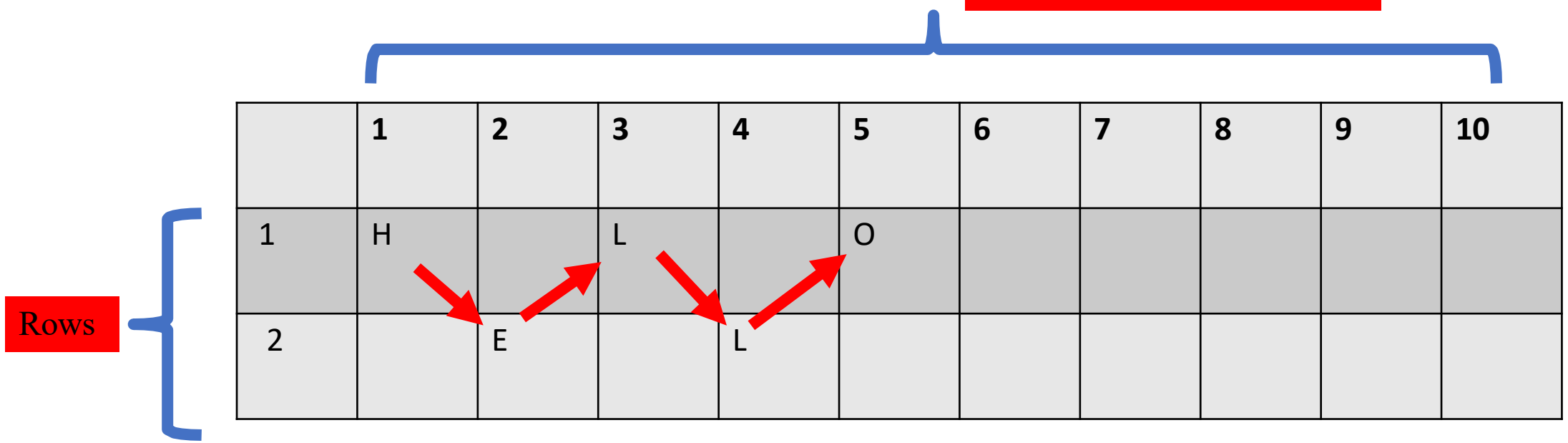
Example: encrypt from PLAIN TEXT TO cipher text

Place **O** in **1st row, 5th column**.

- Message: **"HELLO WORLD“ (A total of 10 letters, so 10 columns.)** depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern

Columns

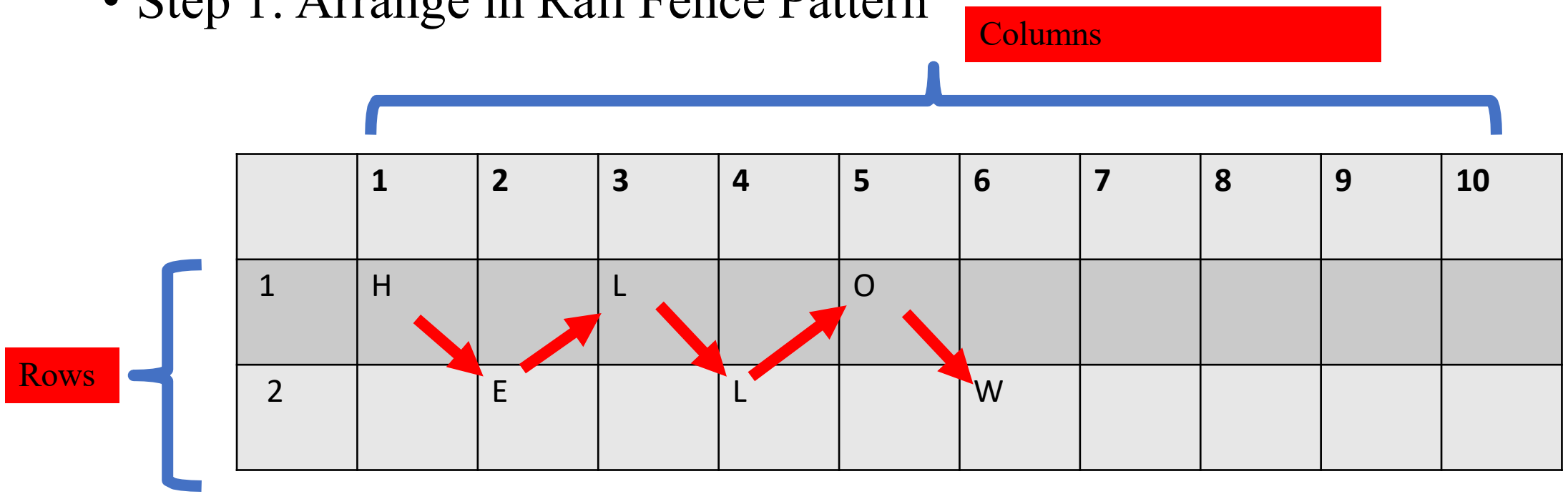
Rows



Example: encrypt from PLAIN TEXT TO cipher text

Place **W** in **2nd row, 6th column**.

- Message: **"HELLO WORLD“ (A total of 10 letters, so 10 columns.)** depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern



Example: encrypt from PLAIN TEXT TO cipher text

- Place **R** in **2nd row, 8th column**.
- Place **L** in **1st row, 9th column**.
- Place **D** in **2nd row, 10th column**.

- Message: **"HELLO WORLD“ (A total of 10 letters, so 10 columns.)** depth : **2 (two row)**
- Step 1: Arrange in Rail Fence Pattern

Columns

Rows

R	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2		E		L		W		R		D

Example:

- Message: **"HELLO WORLD"**
  depth : **2**
- Step 2: **Read Row-wise (Cipher Text)**

ROW 1	**H**		**L**		**O**		**O**		**L**	
ROW 2		E		L		W		R		D

Step 2: Read Row-wise (Cipher Text)

Arrange in Rail Fence Pattern

Row 1: H L O O L

- **Row 2: E L WRD**

ROW 1	**H**		**L**		**O**		**O**		**L**	
ROW 2		E		L		W		R		D

- Cipher Text: **“ROW 1 + ROW 2“**
- **“ H L O O L + E L WRD “**
- **Cipher Text :H L O O L E L WRD**

Rail Fence Cipher – now Decryption Example

- **Cipher text : H L O O L E L WRD**
- **Plaintext : ?**

Decryption Steps

Step 1: Identify Key Matrix
Split the letters based on the answer.
Divide the text into **5 characters per row**.

Step 2: Arrange String in Rail Fence Pattern
Arrange the letters in a **Rail Fence pattern**.
Place the **1st letter in Row 1, Column 1**.
Use the formula to find the next position: **2(depth − 1)**.
Place the **first string and the second string (ending string)** in the pattern.

Step 3: Read Column-wise
Finally, **read the letters column by column** to get the result.

Step 1: Identify Key Matrix

- **Cipher text : H L O O L E L WRD**

- Length of Cipher Text (excluding spaces) = 10
- Depth = 2
- Key Matrix Calculation:

$$\frac{\text{Length of Cipher Text}}{2 \quad \times (\text{Depth} - 1)}$$

Given:

- **Length of Cipher Text = ℓ**
- **Depth (Rails) = n**

Formula to Identify Key Matrix:

$$\frac{\ell}{2 \times (n-1)}$$

Key Matrix Calculation:

- So, we divide into 5 characters per row.

$$\frac{10}{2 \times (2-1)} = \frac{10}{2} = 5$$

Step 1: we divide into 5 characters per row.

- **Cipher text : H L O O L E L WRD**

- Identify the rows : (two strings so two rows)
  Means : We split the string into two parts:
- - Take the first 5 characters from the right side (beginning).
- - Take the next 5 characters from the left side (end).
- Intermediate string will be 2k
-
-

H L O O L



1st string
(1st row)

E L WRD

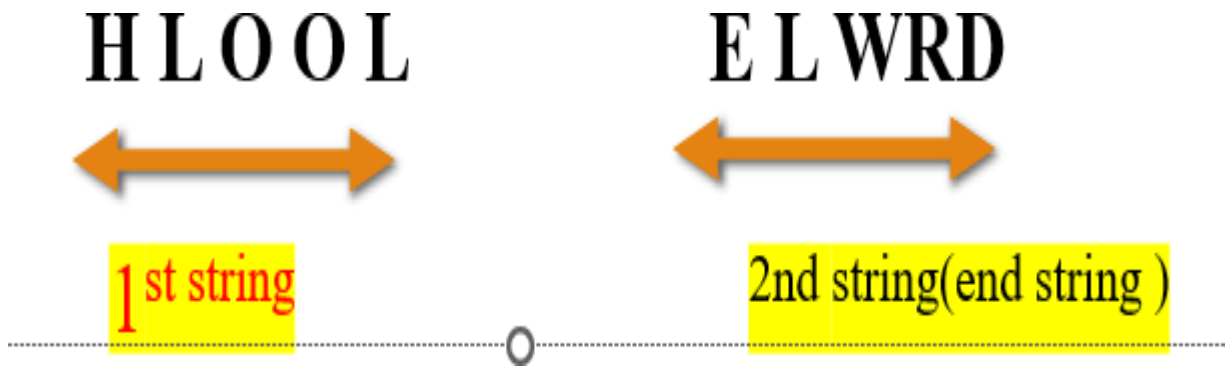


2nd string(end string)
(2nd row)

Step 2: Arrange string in Rail Fence Pattern

- Message: **"H L O O L E L WRD"**
  depth : **2**
- Step 2: Arrange in Rail Fence Pattern

- Place 1st string IN first row :



Step 2: Arrange in Rail Fence Pattern

Take first string in first row and first letter (H)
Depth: 2

Step 2:
Arrange in Rail Fence Pattern
•Place the **1st letter (H)** in **Row 1, Column 1**.



	1	2	3	4	5	6	7	8	9	10
1	H									
2										

Step 2: Arrange in Rail Fence Pattern

Take first string and second letter L
Depth: 2
Step 2: Arrange in Rail Fence Pattern

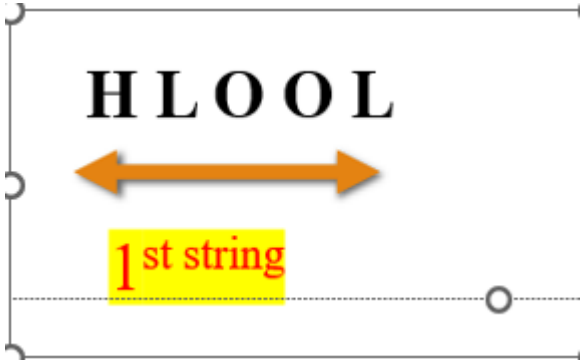
- Use the formula to find the next position:
2(depth − 1) = 2(2 − 1) = 2.

- Move **2 columns forward**.
- Place the **2nd letter (L)** in **Row 1, Column 3**.
- Calculation: **1 + 2 = 3**.

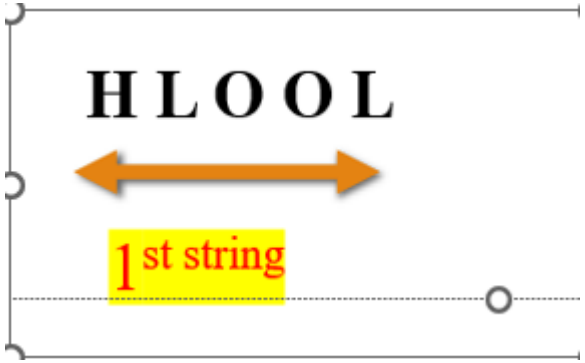


	1	2	3	4	5	6	7	8	9	10
1	H		L							
2										

- The **3rd letter (O)** goes in **Row 1, Column 5**.
- Move **2 columns forward** again.
- Calculation: **3 + 2 = 5**.



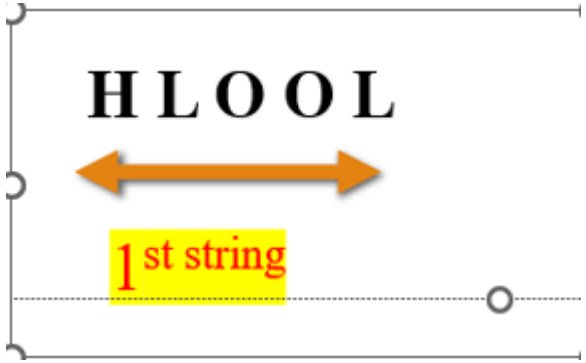
	1	2	3	4	5	6	7	8	9	10
1	H		L		O					
2										



1st string : letter o

- Move **2 columns forward** again.
- Calculation: **5 + 2 = 7**.
- Place **“O”** in **Row 1, Column 7**.

	1	2	3	4	5	6	7	8	9	10
1	H		L		O		O			
2										



Next letter : letter L

- Move **2 columns forward** again.
- Calculation: **7 + 2 = 9**.
- Place **“L”** in **Row 1, Column 9**.

	1	2	3	4	5	6	7	8	9	10
1	H		L		O		O		I	
2										

•Next…… Place 2nd string

Cipher text : **E L WRD**

E L WRD



2nd string(end string)

Step

- Now fill **Row 2**.
- Start from **Column 2**.
- Place the letter **E** in **Row 2, Column 2**.

- After **9**, move **2 places**.
- Next comes **10**.
- Then go to **1**.
- Place **E** in **Column 2 (Row 2)**.

	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2										

Cipher text : **E L WRD**

E L WRD



2nd string(end string)

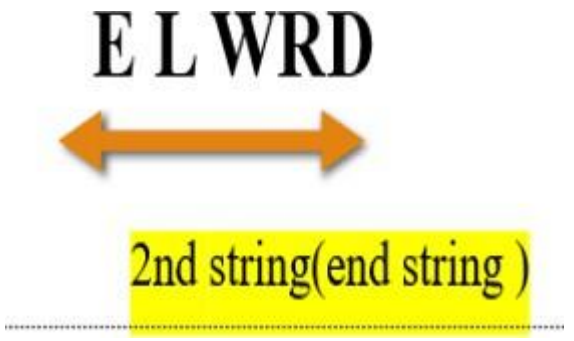
Step

- Now fill **Row 2**.
- Start from **Column 2**.

1. Place the letter **E** in **Row 2, Column 2**.

	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2		E								

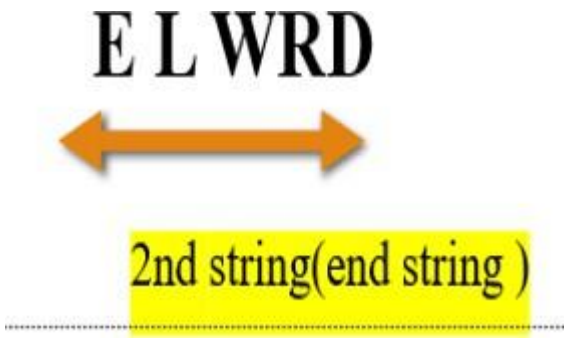
Cipher text : **E L WRD**



- Move **2 columns forward**.
- **2 + 2 = 4**.
- Place **'L'** in **Row 2, Column 4**.

	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2		E		L						

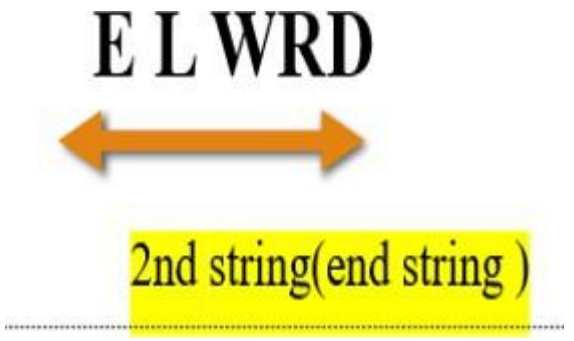
Cipher text : **E L WRD**



- Move **2 columns forward**.
- **4 + 2 = 6**.
- Place **'W'** in **Row 2, Column 6**.

	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2		E		L		W				

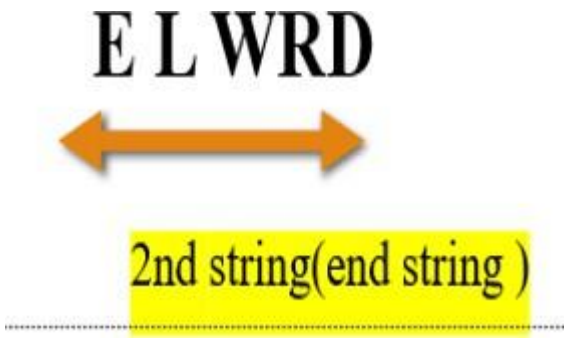
Cipher text : **E L WRD**



- Move **2 columns forward**.
- **6 + 2 = 8**.
- Place **'R'** in **Row 2, Column 8**.

	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2		E		L		W		R		

Cipher text : **E L WRD**



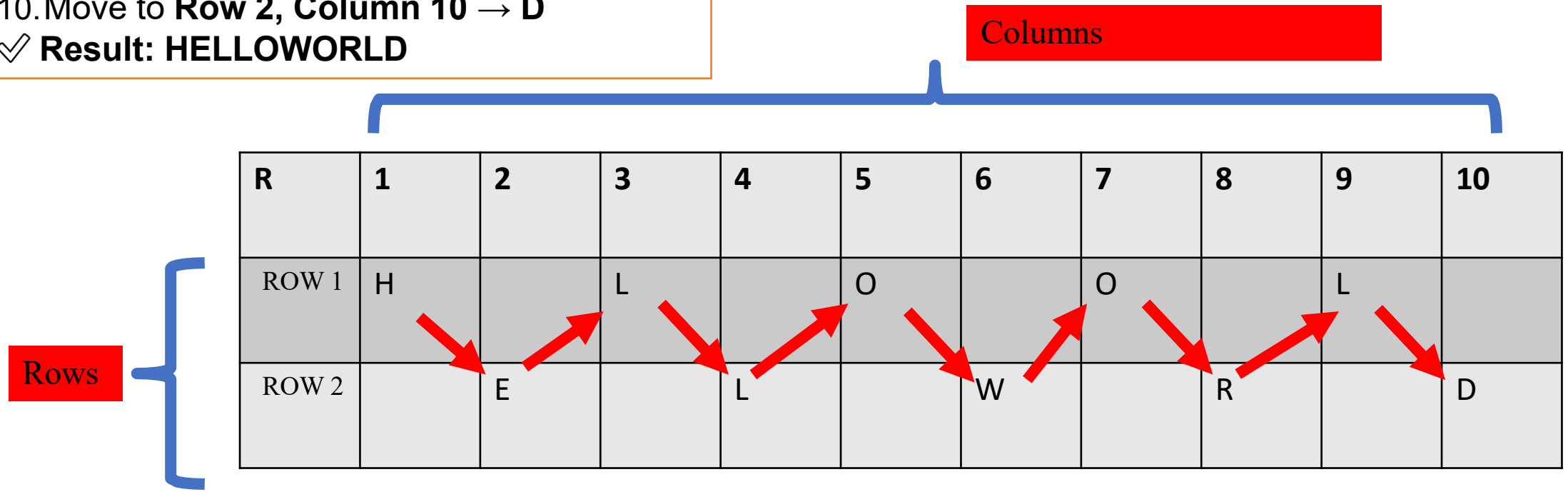
- Move **2 columns forward**.
- **8+ 2 = 10**.
- Place **'D'** in **Row 2, Column 10**.

	1	2	3	4	5	6	7	8	9	10
ROW 1	H		L		O		O		L	
ROW 2		E		L		W		R		D

Read **column-wise (zig-zag)** in simple steps:

1. Start at **Row 1, Column 1 → H**
2. Move to **Row 2, Column 2 → E**
3. Move to **Row 1, Column 3 → L**
4. Move to **Row 2, Column 4 → L**
5. Move to **Row 1, Column 5 → O**
6. Move to **Row 2, Column 6 → W**
7. Move to **Row 1, Column 7 → O**
8. Move to **Row 2, Column 8 → R**
9. Move to **Row 1, Column 9 → L**
10. Move to **Row 2, Column 10 → D**

✓ **Result: HELLOWORLD**



Question :

- Using the Rail Fence Cipher algorithm with a depth of 3, encrypt and decrypt the given text "RAILFENCE".

Encryption steps

Step 1: Choose a Plaintext
- Select a plaintext.
- Letters should be **odd in number**.
- Should **not be a multiple of 2**.
- Example: **"RAILFENCE"**
- Total letters = **9**

Step 2: Use the Formula to Find Required Length
- Use the formula: **Length / 2(depth − 1)**
- Update the **new depth** if needed.

Step 3: Adjust the Text Length
- Add extra letters (like **Z**) if required.
- Make the length suitable for the formula.

Step 4: Fill the Table in Rail Fence Pattern
- Write letters in **zigzag (rail fence) style**.
- Follow the **depth rows**.

Step 5: Read Row-wise
- Read the letters **row by row** to get the result.

Step 1

Step 1: Choose a Plaintext
Start with a **plaintext that has an odd number of letters** and is **not a multiple of 2**.
Example:
Plaintext: **"RAILFENCE"**
•Total letters = **9**
•**9 is an odd number**
•**9 is not a multiple of 2**
So, **"RAILFENCE"** can be used as the plaintext.

Step 2: Use the Formula to Find the Required Length updated new depth

$$\frac{\text{Length of Text}}{2(\text{Depth} - 1)}$$

Substituting the values:

$$\frac{9}{2(3-1)} = \frac{9}{4} = 2.25$$

- We need a **whole number**.
- So, increase the length to **12 letters**.
- Add extra letters like **"Z"**.

Modified Plaintext: "RAILFENCEZZZ"
Total letters: 12.

$$\frac{\text{Length of Text}}{2(\text{depth} - 1)}$$

- **Length of text = 12**
- **Depth = 3**

$$2(3-1) = 2 \times 2 = 4$$

$$\frac{12}{4} = 3$$

Answer: 3

Step 4: Fill the Table in Rail Fence Fashion : row wise

- plaintext: "RAILFENCEZZZ“ and depth =3Now depth is 3 (means 3 rows)

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1												
ROW 2												
ROW 3												

Step 4: Fill the Table in Rail Fence Fashion : row wise : first column

- plaintext: "RAILFENCEZZ“ and fill the FIRST Letter R

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R											
ROW 2												
ROW 3												

Step 4: Fill the Table in Rail Fence Fashion : 2nd column

- plaintext: "RAILFENCEZZZ“ and Move diagonally down to Row 2, Column 2 → "A"

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R											
ROW 2		A										
ROW 3												

Step 4: Fill the Table in Rail Fence Fashion : 3rd column

- plaintext: "RAILFENCEZZZ“ and Move diagonally down to Row 3, Column 3 → "I"

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R											
ROW 2		A										
ROW 3			I									

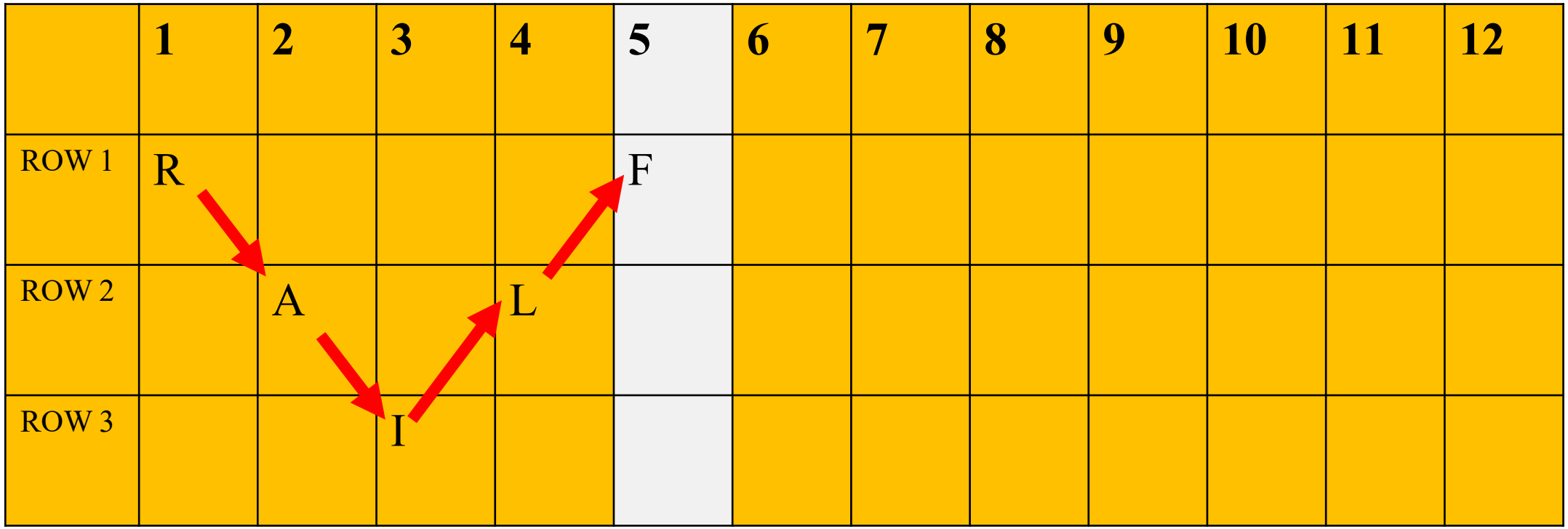
Step 4: Fill the Table in Rail Fence Fashion

- plaintext: "RAILFENCEZZ“ and Move diagonally up to Row 2, Column 4 → "L"

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R											
ROW 2		A		L								
ROW 3			I									

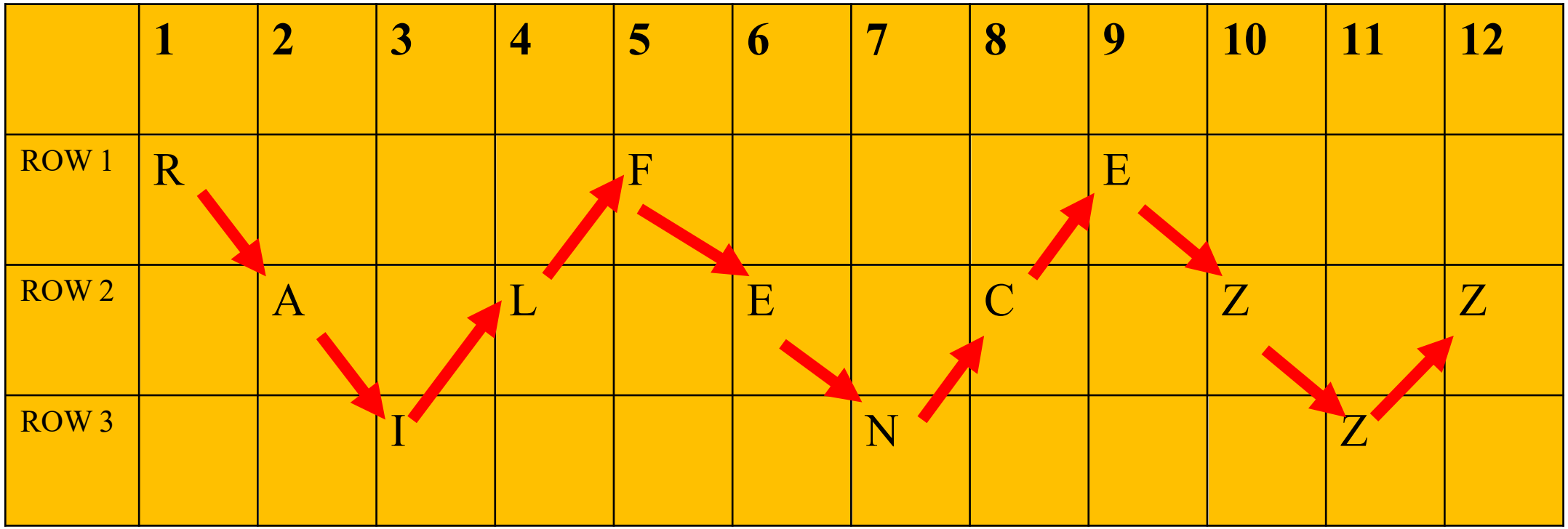
Step 4: Fill the Table in Rail Fence Fashion

- plaintext: "RAILFENCEZZ“ and Move diagonally up to Row 1, Column 5 → "F"



Step 4: Fill the Table in Rail Fence Fashion

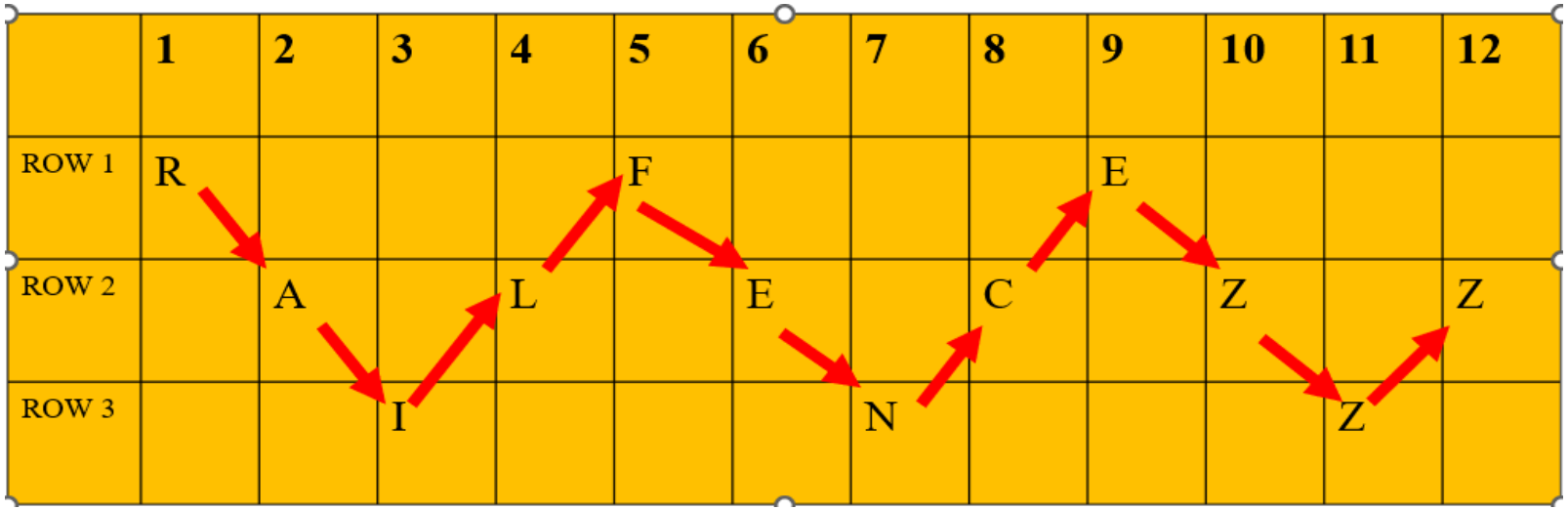
- plaintext: "RAILFENCEZZZ“



- **Move diagonally down to Row 2, Column 6 → "E"**
- **Move diagonally down to Row 3, Column 7 → "N"**
- **Move diagonally up to Row 2, Column 8 → "C"**
- **Move diagonally up to Row 1, Column 9 → “E"**
- **Move diagonally down to Row 2, Column 10 → “Z"**
- **Move diagonally down to Row 3, Column 11 → "Z"**
- **Move diagonally down to Row 3, Column 12→ "Z"**

Step 4: Read in row wise manner

- First Row: RFE
- Second Row: A L E C Z Z
- Third Row: I I L F → "INZ"



Step 5: Read in row wise manner

Cipher text : RFE A L E C Z Z INZ

- Cipher text : row 1 + row 2 + row 3
- : RFE + A L E C ZZ + INZ

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		Z
ROW 3			I				N				Z	

RAIL FENCE Decryption :

- Cipher text : RFE A L E C Z INZ
  plain text : ?

Decryption Steps

<u>Step 1</u>: Identify Key Matrix(identify strings)
Split the letters based on the answer.

<u>Step 2</u>: Arrange String in Rail Fence Pattern
Arrange the letters in a **Rail Fence pattern**.
Place the **1st letter in Row 1, Column 1**.
Use the formula to find the next position: **2(depth − 1)**.
Place the **first string and the second string (ending string)** in the pattern.

<u>Step 3:</u> Read Column-wise
Finally, **read the letters column by column** to get the result.

Step 1: Find the Key Using the Formula

We use the formula to determine the **number of columns** needed:

$$\frac{\text{Length of Text}}{2(\text{Depth} - 1)}$$

Depth =3

Given:

- **Cipher Text** = "RFE A L E C Z Z INZ" (12 letters)
- **Depth** = 3

Substituting values:

$$\text{Key (K)} = \frac{12}{2(3-1)} = \frac{12}{4} = 3$$

So, the **key (number of full zigzag cycles) = 3**.

Step 1 Identify the Distribution of Letters

From **Step 1**, we determined that the text length is **12** and the depth is **3**.

Now, we split the **cipher text**:

Cipher Text: "**RFE A L E C Z Z INZ**"

Breakdown Based on Rail Fence Structure:

- First Row (First 3 letters) → "RFE"
- Middle Row (2 × Depth = 6 letters) → "ALECZZ"
- Last Row (Last 3 letters) → "INZ"

Middle row : 2K

Distribution of Letters (Splitting of letters)

RFE A L E C Z Z INZ



1ST (Middle: 2nd row) LAST string

- Intermediate string will be 2*key =2*3 = 6 characters

Step 2: Place Letters in Row 1

Cipher text : RFE A L E C ZZ INZ



1st row

We will now fill **only the first row** in the Rail Fence pattern.

Steps to Place Letters in Row 1:

1.Place the 1st letter (R) in Row 1, Column 1.

2. **Use the formula:**

$$2(\text{Depth} - 1) = 2(3 - 1) = 4$$

So, move **4 columns forward** to place the next letter.

3. **Place the 2nd letter (F) in Row 1, Column 5.**

4. **Move 4 columns forward again** (5 + 4 = 9).

5. **Place the 3rd letter (E) in Row 1, Column 9.**

Step 3: Place Letters in Row 1 place first letter of 1st row : R

- Cipher text : row 1
- : RFE

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R											
ROW 2												
ROW 3												

Step 3: Place Letters in Row 1

next letter F

Cipher Text – Row 1

1. Start at **Row 1, Column 1** → place **R**.
2. Move **4 columns forward** → **1 + 4 = 5**.
3. Place the **2nd letter (F)** in **Row 1, Column 5**.
4. Next letter **E** goes in **Row 1** following the pattern.
5. So Row 1 becomes: **R _ _ _ F _ _ _ E**.
6. Read the letters → **RFE**.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F							
ROW 2												
ROW 3												

Step 3: Place Letters in Row 1

- **Cipher text – Row 1**
- Place the **3rd letter (E)** in **Row 1, Column 9**.
- Move **4 columns forward again**.
- Calculation: **5 + 4 = 9**.
- Row 1 letters become: **R F E**.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2												
ROW 3												

Next fill 2nd row

- **After placing one letter**, leave **one column gap**.
- Do **not fill the next column**.
- Move to the **next column after the gap**.
- Then **place the next letter** there.

RFE ALECZZ INZ



1ST (Middle: 2nd row) LAST string

Step 4: Fill Row 2
Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Move to Column 2** (Column 1 already has **R**).
- Place the letter **"A"** in **Row 2, Column 2**.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A										
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 3**, then move to Column 4 → Place **"L"** in **Row 2, Column 4**.

- **After placing one letter**, leave **one column gap**.
- Do **not fill the next column**.
- Move to the **next column after the gap**.
- Then **place the next letter** there.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L								
ROW 3												

Step 4: Fill Row 2
Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 5**, then move to Column 6

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L								
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 5**, then move to Column 6 → Place **"E"** in **Row 2, Column 6**.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E						
ROW 3												

Step 4: Fill Row 2
Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 7**, then move to Column 8

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E						
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 7**, then move to Column 8 : Place **"C"** in **Row 2, Column 8**.

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
ROW 1	R				F				E			
ROW 2		A		L		E		C				
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 9**, then move to Column 10

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C				
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

•

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

•

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: **"A L E C Z Z"**.

- **Skip Column 11**, then move to Column 12→
  Place **"Z"** in **Row 2, Column 12**.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		Z
ROW 3												

Step 4: Fill Row 2

Now, we will fill **Row 2** using the middle set of letters: "**A L E C Z Z**".

Steps to Place Letters in Row 2:

1. **Move to Column 2** (since Column 1 is already filled with "R") → Place "**A**" in **Row 2, Column 2**.
2. **Skip Column 3**, then move to Column 4 → Place "**L**" in **Row 2, Column 4**.
3. **Skip Column 5**, then move to Column 6 → Place "**E**" in **Row 2, Column 6**.
4. **Skip Column 7**, then move to Column 8 → Place "**C**" in **Row 2, Column 8**.
5. **Skip Column 9**, then move to Column 10 → Place "**Z**" in **Row 2, Column 10**.
6. **Skip Column 11**, then move to Column 12 → Place "**Z**" in **Row 2, Column 12**.

Step 5: Fill Row 3

Now, we will fill **Row 3** using the last set of letters: "**I N Z**".

RFE A L E C Z Z INZ



1ST (Middle: 2nd row) LAST string

Steps to Place Letters in Row 3:

1. **Start from Column 3** (since Columns 1 and 2 are already filled) → Place "**I**" in **Row 3, Column 3**.
2. **Use the formula:**

$$2(\text{Depth} - 1) = 2(3 - 1) = 4$$

So, move **4 columns forward** (3 + 4 = 7).

3. **Place "N" in Row 3, Column 7.**
4. **Move 4 columns forward again** (7 + 4 = 11).
5. **Place "Z" in Row 3, Column 11.**

Same rule of 1st row

Step 5: Fill Row 3

Now, we will fill **Row 3** using the middle set of letters: **“INZ"**.

- **Start from Column 3** (since Columns 1 and 2 are already filled) → Place **"I"** in **Row 3, Column 3**.

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		Z
ROW 3			I									

Step 5: Fill Row 3

Now, we will fill **Row 3** using the middle set of letters: **“INZ"**.

- **move 4 columns forward (3 + 4 = 7). Place "N" in Row 3, Column 7.**

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		Z
ROW 3			I				N					

Step 5: Fill Row 3
Now, we will fill **Row 3** using the middle set of letters: **“INZ"**.

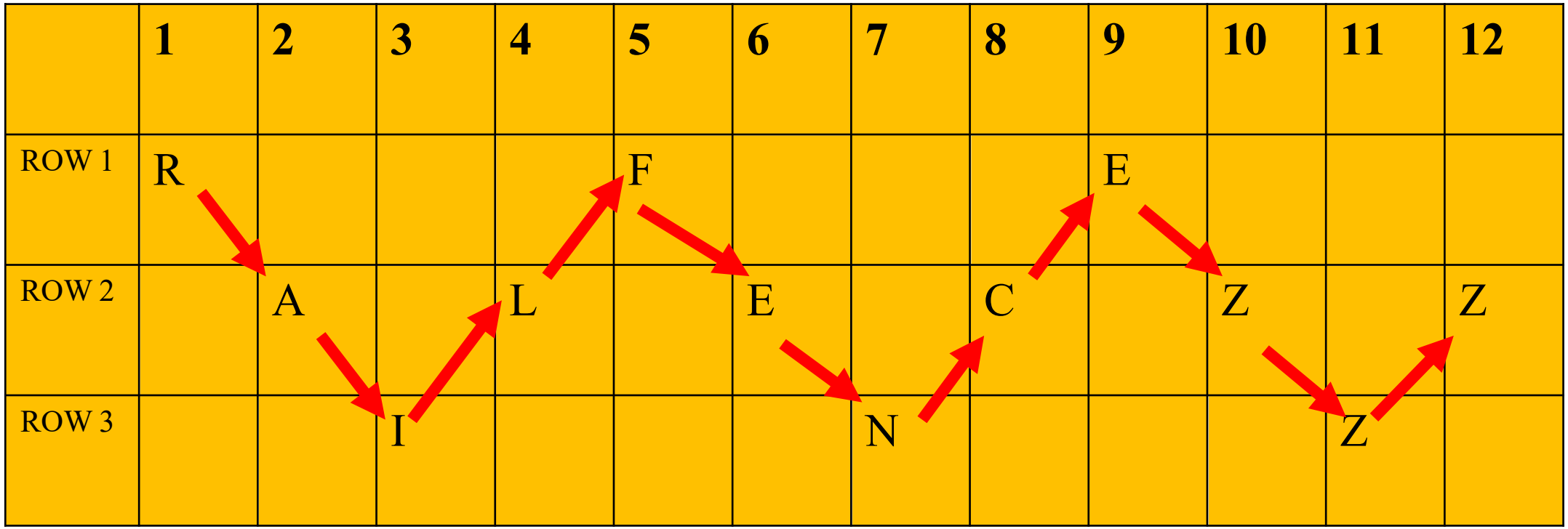
- **Move 4 columns forward again (7 + 4 = 11).Place "Z" in Row 3, Column 11.**

	1	2	3	4	5	6	7	8	9	10	11	12
ROW 1	R				F				E			
ROW 2		A		L		E		C		Z		Z
ROW 3			I				N				Z	

1. **Column 1** → R
2. **Column 2** → A
3. **Column 3** → I
4. **Column 4** → L
5. **Column 5** → F
6. **Column 6** → E
7. **Column 7** → N
8. **Column 8** → C
9. **Column 9** → E
10. **Column 10** → Z
11. **Column 11** → Z
12. **Column 12** → Z

Step 4: Fill the Table in Rail Fence Fashion

- plaintext: "RAILFENCEZZZ“



Final Decrypted Text:
"RAILFENCEZZZ"

(Remove the extra **"Z"** if it was added for adjustment)

Original Plaintext: "RAILFENCE"

- **Decryption is complete**

Summary :

Task Table:

Task	Description
1. Encryption	Encrypt the plaintext "**RAILFENCE**" using Rail Fence Cipher with a depth of **3**.
2. Adjust Length	Since the text length (9) is not a multiple of 2, **add extra letters** ("**ZZZ**") to make it **12**.
3. Arrange in Zigzag	Fill the letters in a **zigzag pattern** and display in a table.
4. Read Row-Wise	Extract the **cipher text** from the table.
5. Decryption	Use the **cipher text**: "**RFE ALECZZ INZ**" to reconstruct the original message.
6. Read Column-Wise	Recover the plaintext by reading the table column-wise.

2.Row-by-row and Column-by-column Method

- **Encryption:**
  Write → **Row-wise**
  Read → **Column-wise**
- **Decryption:**
  Write → **Column-wise**
  Read → **Row-wise**

Example: Plaintext = COMPUTER

- **Step 1: Make the Matrix**
- Use **3 columns**.

Row/column➔	Column 1	Column 2	Column 3
1	C	O	M
2	P	U	T
3	E	R	—

Step 2: Write Row-by-Row

- **Row 1:** C → O → M
  Row 2: P → U → T
  Row 3: E → R → —

Row/column →	Column 1	Column 2	Column 3
1	C	O	M
2	P	U	T
3	E	R	–

Step 3: Read Column-by-Column

- **Column 1:** C → P → E
- **Column 2:** O → U → R
- **Column 3:** M → T
- **Step 4: Ciphertext**
- **C P E O U R M T**
- **Final Answer**
- **Plaintext:** COMPUTER
  Ciphertext: CPEOURMT ✓

Row/column →	Column 1	Column 2	Column 3
1	C	O	M
2	P	U	T
3	E	R	–

Decryption – Row-by-Row and Column-by-Column

- **Given:**
- **Ciphertext:** CPEOURMT
- **Columns:** 3
- **Rows:** 3

Step 1: Make the Matrix



Row/column →	C1	C2	C3
1	C	O	M
2	P	U	T
3	E	R	—

- **Step 2: Put Ciphertext Column-by-Column**
- **Column 1:** C → P → E
- **Column 2:** O → U → R
- **Column 3:** M → T



Row/column →	C1	C2	C3
1	C	O	M
2	P	U	T
3	E	R	—

- **Step 3: Read Row-by-Row**
- **Row 1:** C O M
- **Row 2:** P U T
- **Row 3:** E R
- **Final Answer**
- **Ciphertext:** CPEOURMT
  Plaintext: COMPUTER ✓



Row/column →	C1	C2	C3
1	C	O	M
2	P	U	T
3	E	R	—

Keyed Transposition Ciphers

- → Permutation Key

- A **keyed transposition cipher** rearranges the positions of letters.
- The letters themselves are **not changed**.
- A **permutation key** tells us the new order.
- The same key is used for every block.

Example :

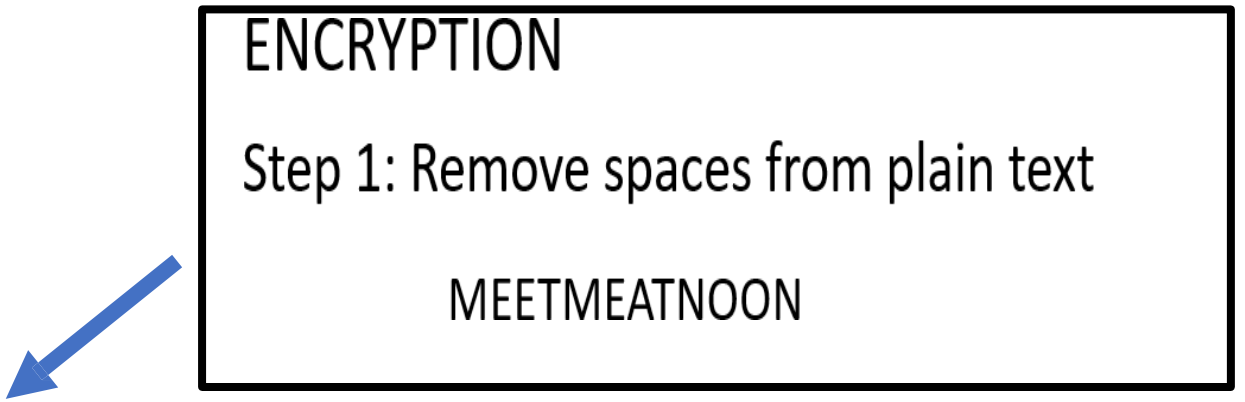
- Plaintext: MEET ME AT NOON
- Key : 2 4 1 3 5
- Block size = 5

ENCRYPTION

Step 1: Remove spaces from plain text

MEETMEATNOON

After remove spaces ,divide plain text into block



Step 2: Divide into blocks of 5

5 letters 5letters 2 letters

- MEETM | EATNO | ON

last block has only 2 letters, so add Z as padding.

5 letters 5 letters 5 letters

MEETM | EATNO |ONZZZ

So the blocks are:

- 5 letters 5 letters 5 letters
- MEETM |EATNO | ONZZZ

Step 3: Apply the permutation key

Key: 2 4 1 3 5

2nd character → 1st
4th character → 2nd
1st character → 3rd
3rd character → 4th
5th character → 5th

Step 4: Encrypt first block MEETM

Take:
2nd = E 4th = T 1st = M 3rd = E 5th = M

Therefore:
MEETM →ETMEM

position	**1**	**2**	**3**	**4**	**5**
Plain text	M	E	E	T	M
key	2	4	1	3	5
Reposition Text ,BASED ON KEY	E	T	M	E	M

Step 5: Encrypt second block
EATNO

2nd = A
4th = N
1st = E
3rd = T
5th = O

Therefore:
EATNO →ANETO

position	1	2	3	4	5
Plain text	E	A	T	N	O
key	2	4	1	3	5
Reposition Text ,BASED ON KEY	A	N	E	T	O

Step 5: Encrypt Third block

ONZZZ

Take:
2nd = N 4th = Z 1st = O 3rd = O 5th = Z
Therefore:
ONZZZ →NZOZZ

position	1	2	3	4	5
Plain text	O	N	Z	Z	Z
key	2	4	1	3	5
Reposition Text ,BASED ON KEY	N	Z	O	z	Z

Final Cipher text

1st block	2nd block	3rd block
ETMEM	| ANETO	| NZOzZ

Therefore:

Ciphertext = ETMEMANETONZOZZ

DECRYPTION

Step 1: Divide cipher text into blocks of 5

5 letter **5 letter** **5 letter**

ETMEM | ANETO | NZOZZ

Step 2: Use the reverse permutation

Original key:
2 4 1 3 5

☆ **Easy Trick find original key ➔ write position**

Find 1 → write its position (3)
Find 2 → write its position (1)
Find 3 → write its position (4)
Find 4 → write its position (2)
Find 5 → write its position (5)

ORIGINAL position	1	2	3	4	5
Orginal key	2	4	1	3	5
Reverse key Mention position	3	1	4	2	5

Step 3: Decrypt first block : ETMEM → MEETM

3rd position is = M
1st = E
4th = E
2nd = T
5th = M

Original position	**1**	**2**	**3**	**4**	**5**
Plain text	E	T	M	E	M
key	3	1	4	2	5
Key map with original position	M	E	E	T	M

Step 4: Decrypt second block

ANETO →EATNO

Original position	**1**	**2**	**3**	**4**	**5**
Plain text	A	N	E	T	O
key	3	1	4	2	5
Key map with original position	E	A	T	N	O

Step 5: Decrypt third block

NZOOZ → ONzZZ

Original position	**1**	**2**	**3**	**4**	**5**
Plain text	N	Z	O	z	Z
key	3	1	4	2	5
Key map with original position	O	N	z	Z	Z

Step 6 : Combine and get final message (Decryption)

Step 6.1 : Combine all decrypted blocks.

(5 letters) MEETM | (5 letters) EATNO | (5 letters) ONZZZ

MEETM EATNO ONZZZ

Each block has 5 letters (same as key size).

Step 6.2 : Remove the padding letters (ZZZ).

MEETM EATNO ONZZZ → Padding letters (added during encryption)

↓

MEETM EATNO ON Remove ZZZ

Step 6.3 : Join the letters (remove spaces).

MEETM EATNO ON

↓

MEETMEATNOON

Step 6.4 : Add spaces to get the original message.

MEETMEATNOON

↓

MEET ME AT NOON

Step 6.5 : Final Answer :

Plaintext (after decryption) =

MEET ME AT NOON

We add spaces based on the meaning of the message. (There is no fixed formula for spaces.)

TRANSPOSITION CIPHERS

↓

1. Keyless
   - Rail Fence
   - Row → Column / Column → Row

↓

2. Keyed
   - Permutation Key

   Example

↓

3. Combining Two Approaches
   - Keyed Columnar Transposition

   Example

Keyed Columnar Transposition.

This method combines:

Keyless transposition
Keyed transposition
It is also called **Keyed Columnar Transposition**.

Why Padding? ⭐

Padding means **adding extra letters** at the end of plaintext.

Main reason

In our example:

- Key has **5 columns**
- So plaintext should fit properly into **groups of 5**

Plaintext:

`ENEMYATTACKSTONIGHT`

Count = **19 letters**

But:

19 ÷ 5 (key) = 5)19(3
15

4

3 remainder 4

→ **Remainder = 4**

→ We get a **complete integer quotient (3)**.

Without padding

❌ One empty place.

3	1	4	5	2
E	N	E	M	Y
A	T	T	A	C
K	S	T	O	N
I	G	H	T	—

With padding
Add **Z**:
ENEMYATTACKSTONIGHTZ
Now = **20 letters**
20 ÷ 5 (key)= 4 (complete integer)

So we get a complete table:

3	1	4	5	2
E	N	E	M	Y
A	T	T	A	C
K	S	T	O	N
I	G	H	T	Z

Remember 🧠
Padding = extra letter added to fill the empty space.
Usually Z is used as a **dummy/padding character** in these examples.

Example
Key = **5 columns**
Plaintext = `HELLOWORLD`
Count = **10**
`10 ÷ 5 = 2` → ✅ No padding.

Plaintext = `HELLOWOR`
Count = **8**

8 ÷ 5 = (Q)1 AND remainder 3

$$\frac{8}{5} = \text{fraction, so add 2 letters } 10/5 = 2$$

➡️ Add **2 Z's**
`HELLOWORZZ`
Now = **10 letters** → ✅ Complete table.

Combining Two Approaches

Keyless + Keyed Transposition

Step 1: Plaintext

ENEMY ATTACKS TONIGHT

Remove spaces:
ENEMYATTACKSTONIGHT

Add Z as padding:
ENEMYATTACKSTONIGHTZ

Step 2:
Key
Key = **3 1 4 5 2**
Number of columns = **5**
Write plaintext **row-wise**:

3	1	4	5	2
E	N	E	M	Y
A	T	T	A	C
K	S	T	O	N
I	G	H	T	Z

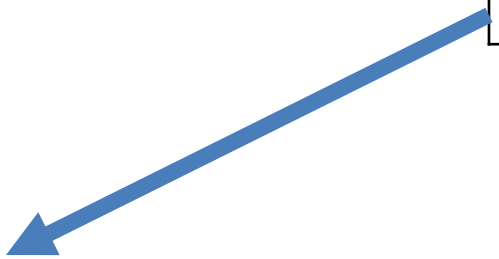
Step 3: Arrange columns using key

Read columns in key order:

1 → 2 → 3 → 4 → 5

- Key **1** → column 2
- Key **2** → column 5
- Key **3** → column 1
- Key **4** → column 3
- Key **5** → column 4

3	1	4	5	2
E	N	E	M	Y
A	T	T	A	C
K	S	T	O	N
I	G	H	T	Z



1	2	3	4	5
N	Y	E	E	M
T	C	A	T	A
S	N	K	T	O
G	Z	I	H	T

New order:

Step 4: Read column-wise

Read from top to bottom:

- Column 1 → `NTSG`
- Column 2 → `YCNZ`
- Column 3 → `EAKI`
- Column 4 → `ETTH`
- Column 5 → `MAOT`

1	2	3	4	5
N	Y	E	E	M
T	C	A	T	A
S	N	K	T	O
G	Z	I	H	T

Final Ciphertext

`NTSGYCNZEAKIETTHMAOT`

Decryption — Stepwise 🔓

Using our example:

Ciphertext:

NTSGYCNZEAKIETTHMAOT

Key: 3 1 4 5 2

Columns: 5

Rows: 4

Step 1: Divide ciphertext

20 letters ÷ 5 columns = **4 letters per column**

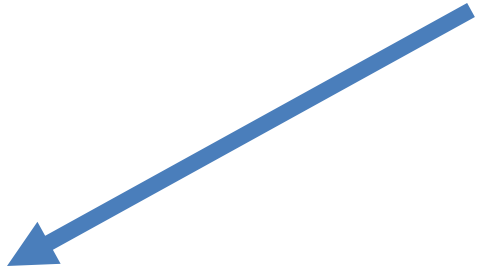
```
NTSG | YCNZ | EAKI | ETTH | MAOT
```

Step 2: Arrange according to key

To Original key:

3 1 4 5 2

1	2	3	4	5
N	Y	E	E	M
T	C	A	T	A
S	N	K	T	O
G	Z	I	H	T



3	1	4	5	2
E	N	E	M	Y
A	T	T	A	C
K	S	T	O	N
I	G	H	T	Z

Step 3: Read row-wise

E N E M Y

A T T A C

K S T O N

I G H T Z

➡️ ENEMYATTACKSTONIGHTZ

🧠 Decryption Trick
Divide → Place columns → Read rows → Remove padding → Add spaces

Step 4: Remove padding

Last letter **Z** = padding.

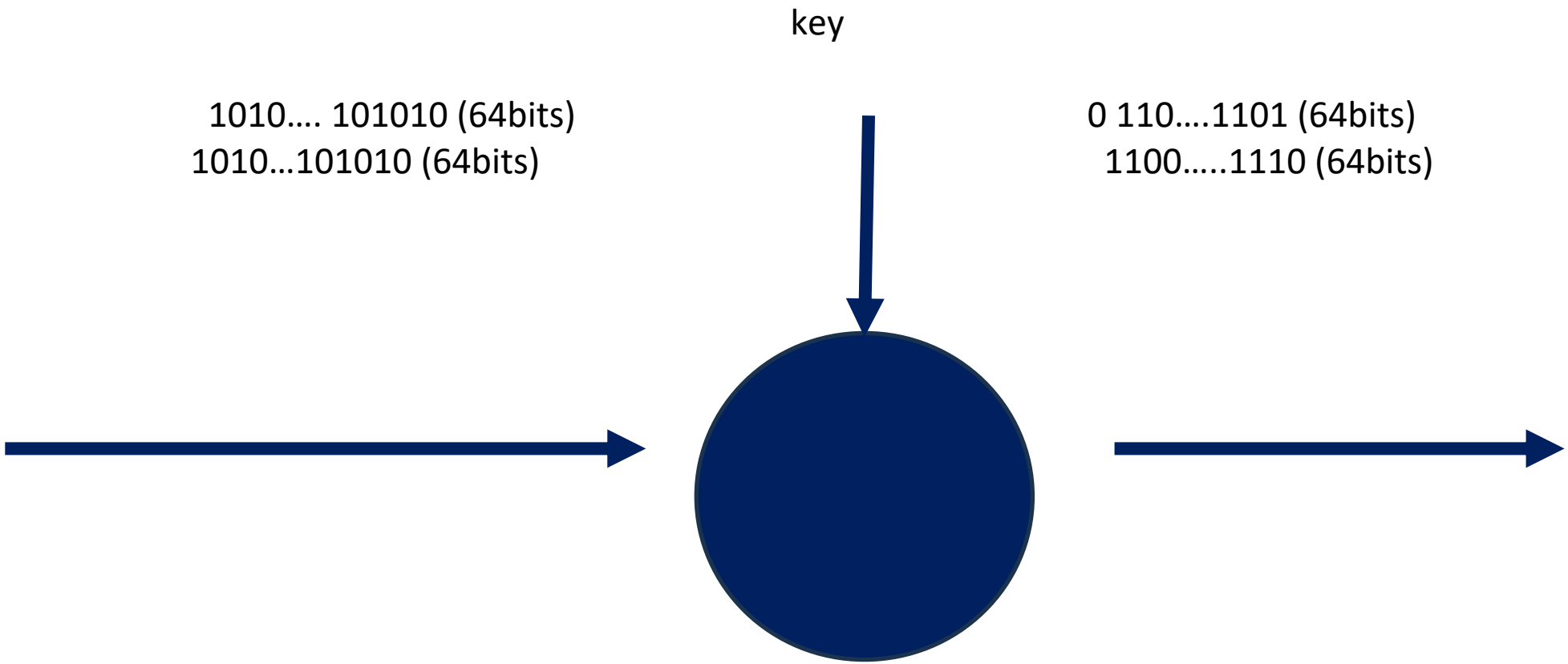
ENEMYATTACKSTONIGHT

Step 5: Restore spaces

ENEMY ATTACKS TONIGHT

Block Cipher

Diagram : key stream generator



Confusion vs Diffusion

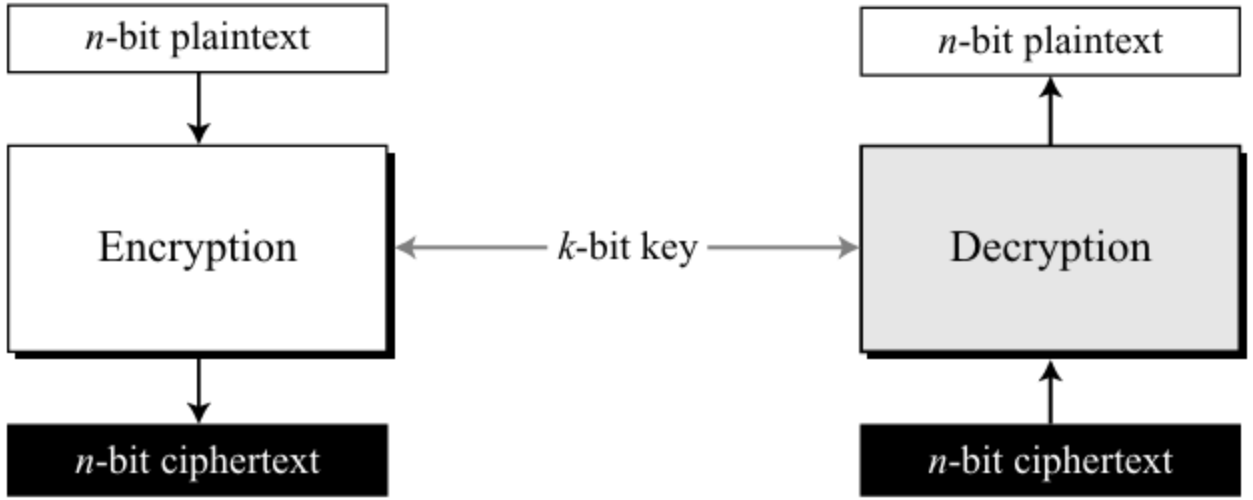
Confusion: Hides the relationship between **key and ciphertext**.

Diffusion: Spreads plaintext information across **ciphertext**.

Modern Block Ciphers

Works with **bits (0 and 1)**.
Encrypts data in **fixed-size blocks**.
Uses a **secret key**.
Same key is used for encryption and decryption.
Common block sizes: **64, 128, 256, 512 bits**.

Figure 5.1 *A modern block cipher*



Padding Calculation

Book Example 5.1 ⭐

Given:

- 100 characters
- 8-bit ASCII
- Block size = 64 bits

Step 1: Calculate total bits

100 bytes × 8 = 800 bits

Step 2: Find the remainder

800 mod 64 = 32 bits

Step 3: Calculate padding

Padding = 64 − 32
Padding = 32 bits

Final Answer:

Padding = 32 bits
Total = 800 + 32 = 832 bits
Blocks = 832 ÷ 64 = 13 blocks

1. Components of Modern Block Ciphers

1. P-Box
2. S-Box
3. XOR
4. Circular Shift
5. Swap
6. Split
7. Combine

P-Box (Permutation Box)

- It is used in **block ciphers**.
- It changes the **positions of bits**.
- Bit values **do not change**.
- Main purpose: **Diffusion**.

Figure 5.4 *Three types of P-boxes*

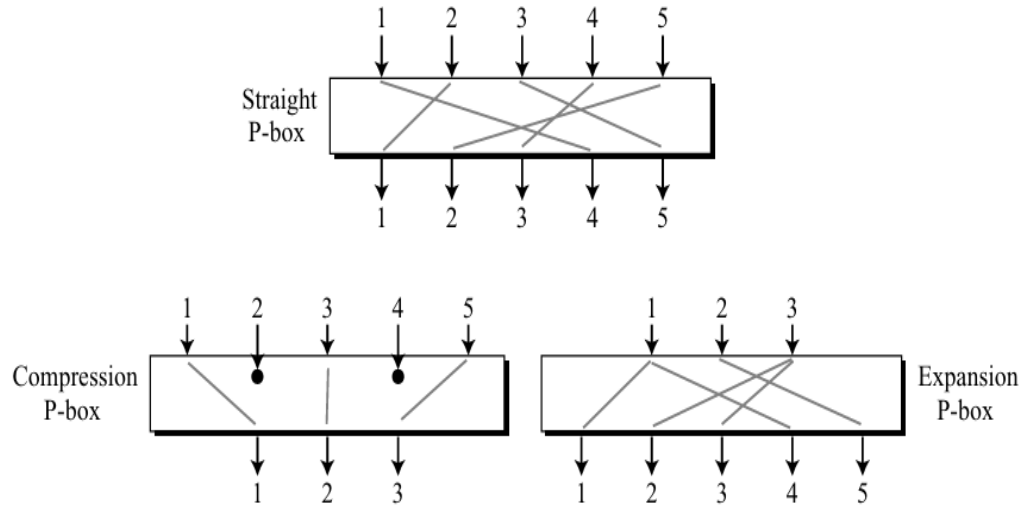


Figure 5.4 shows a 5 × 5 straight P-box, a 5 × 3 compression P-box, and a 3 × 5

Types of P-Box

1. Straight P-Box
2. Expansion P-Box
3. Compression P-Box

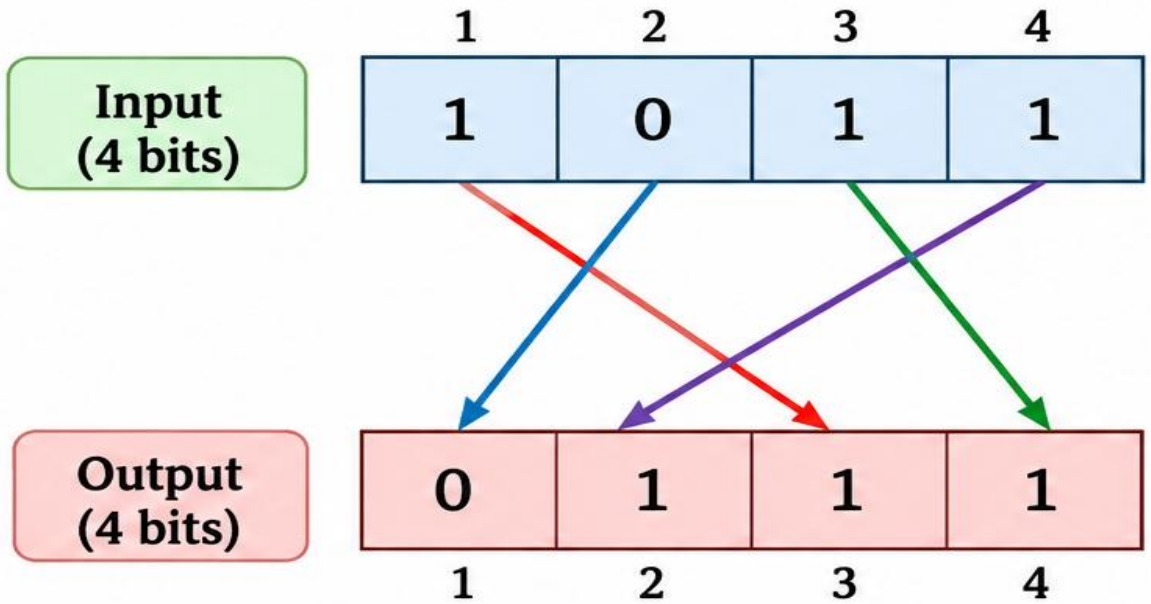
Straight P-Box (Example)

- Input bits = Output bits.
- Bits are only **shuffled** (permuted).
- No bit is added or removed.

Example:
4-bit Straight P-Box

Input position (i)	1	2	3	4
Output position $P(i)$	3	1	4	2

This means: the bit at position 1 goes to 3, 2 goes to 1, 3 goes to 4, and 4 goes to 2.



For input 1 0 1 1 :

- Output bit 1 ← Input bit 2 (0)
- Output bit 2 ← Input bit 4 (1)
- Output bit 3 ← Input bit 1 (1)
- Output bit 4 ← Input bit 3 (1)

So, Output = 0 1 1 1

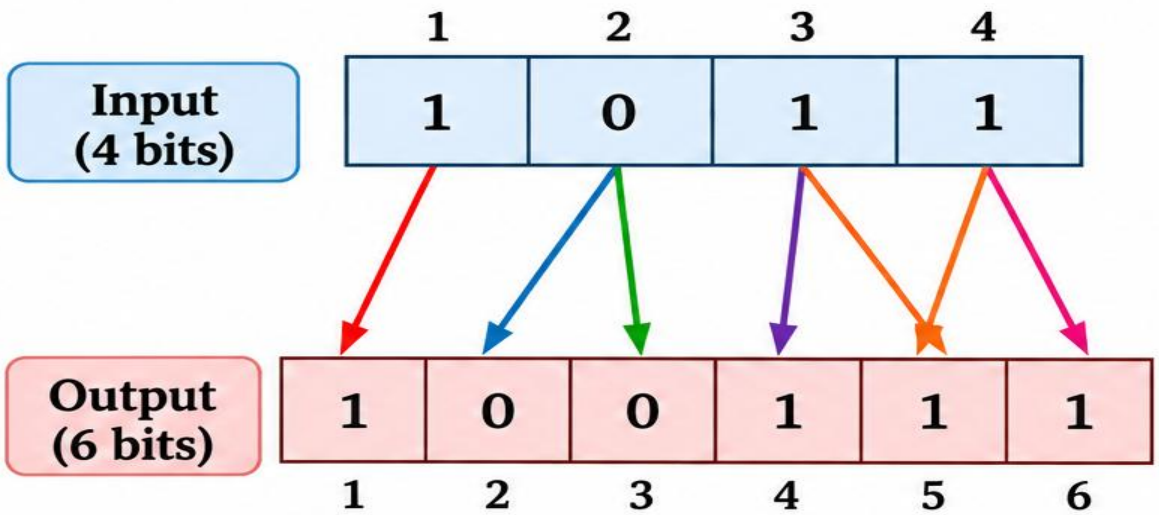
Same number of bits in input and output.
Only the positions are changed (shuffled).

Expansion P-Box (Example)

- Output bits > Input bits.
- Some bits are **duplicated**.
- $N_{in} < N_{out}$

Example:
4-bit to 6-bit Expansion P-Box

Output position (i)	1	2	3	4	5	6
Input position P(i)	1	2	2	3	4	4



For input 1 0 1 1 :

- Output bit 1 ← Input bit 1 (1)
- Output bit 2 ← Input bit 2 (0)
- Output bit 3 ← Input bit 2 (0) **[duplicated]**
- Output bit 4 ← Input bit 3 (1)
- Output bit 5 ← Input bit 4 (1)
- Output bit 6 ← Input bit 4 (1) **[duplicated]**

So, Output = 1 0 0 1 1 1

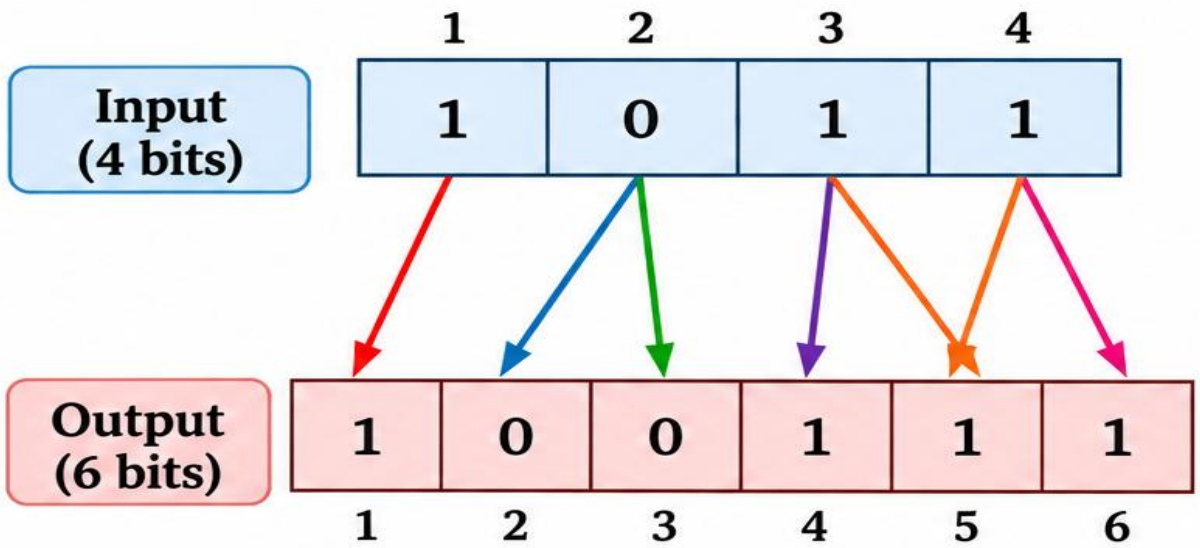
Number of input bits (4) < Number of output bits (6).
Some input bits are repeated (duplicated) to create a larger output.

Expansion P-Box (Example)

- Output bits > Input bits.
- Some bits are **duplicated**.
- $N_{in} < N_{out}$

Example:
4-bit to 6-bit Expansion P-Box

Output position (i)	1	2	3	4	5	6
Input position P(i)	1	2	2	3	4	4



For input 1 0 1 1 :

- Output bit 1 ← Input bit 1 (1)
- Output bit 2 ← Input bit 2 (0)
- Output bit 3 ← Input bit 2 (0) **[duplicated]**
- Output bit 4 ← Input bit 3 (1)
- Output bit 5 ← Input bit 4 (1)
- Output bit 6 ← Input bit 4 (1) **[duplicated]**

So, Output = 1 0 0 1 1 1

Number of input bits (4) < Number of output bits (6).
Some input bits are repeated (duplicated) to create a larger output.

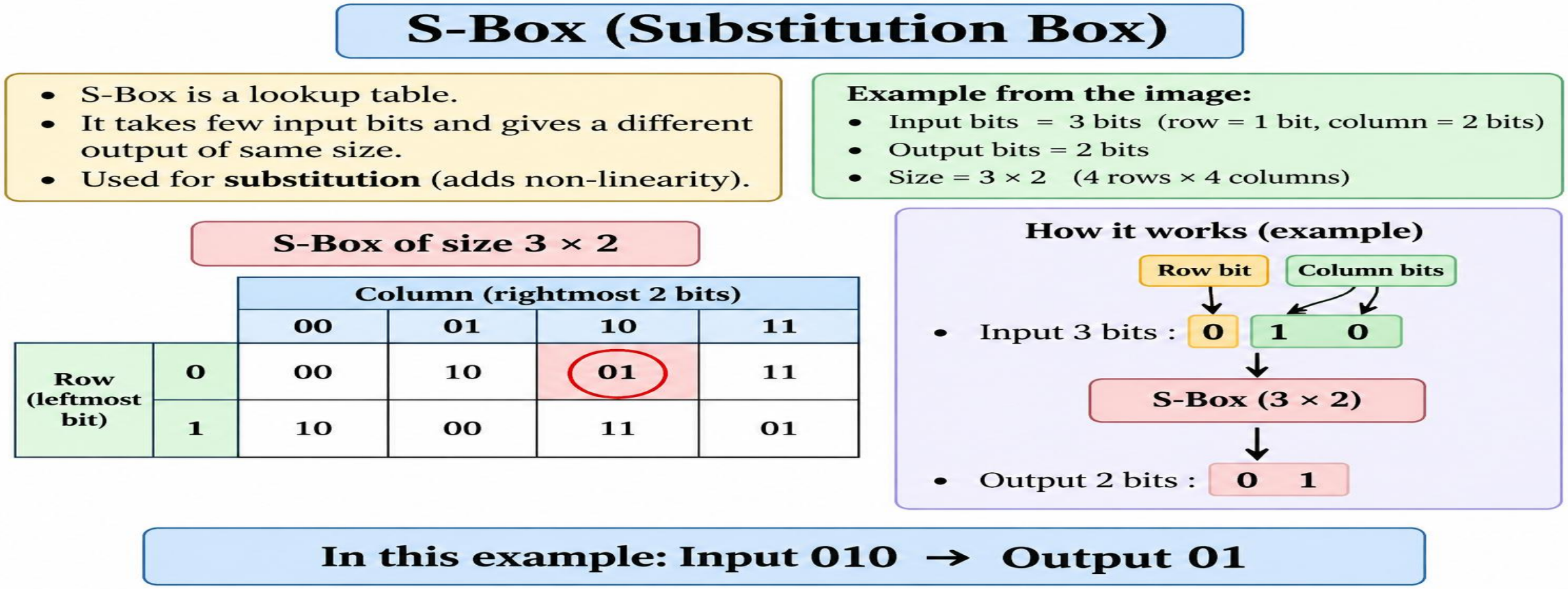
S-Box (Substitution Box) ⭐

It is used in **modern symmetric-key ciphers**.

It changes the **input bits into output bits**.

It uses a **pre-defined table**.

It provides **confusion**.



S-Box Example (6-bit to 4-bit)

- Input = 6 bits
- Output = 4 bits
- Row bits = **first and last bit** (b1 and b6)
- Column bits = **middle 4 bits** (b2 b3 b4 b5)
- Output = value from the table

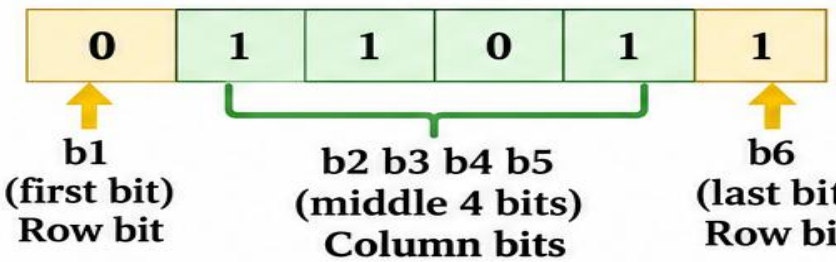
Example: Input 0 1 1 0 1 1

- First bit (b1) = 0
- Last bit (b6) = 1 → Row = 01 (decimal 1)
- Middle 4 bits (b2 b3 b4 b5) = 1 1 0 1 → Column = 1101 (decimal 13)
- From table at row 01 and column 1101 = 1100
- Output = 1 1 0 0

S-Box S1 (6-bit → 4-bit)

Row (b1 b6)	Column (b2 b3 b4 b5)																
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1001	0101	1100	1101	1110	1111
00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0100	1111	1101	0001	0011	0010	1000	1101
01	1110	1011	0010	1100	0100	0111	1101	0001	0100	1110	1011	1000	0101	1100	**1100**	1000	0111
10	0100	0010	0001	1011	1010	1101	0111	1000	1101	0111	0001	0100	1011	1011	1000	0110	0010
11	1011	1000	1100	0111	0001	1110	0010	1101	0010	1101	0100	1011	0110	0010	0001	0010	0100

Input 6 bits : 0 1 1 0 1 1



S-Box (6 → 4)

Output 4 bits : 1 1 0 0

1	1	0	0

So, input 011011 → output 1100

xOR Operation

XOR = Exclusive OR

- Used in **modern block ciphers**.
- Combines **plaintext/data with key**.
- Same bits → **0**
- Different bits → **1**

XOR Table

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

Example

```
Data  :  1011
Key   :  1100
         ----
XOR   :  0111
```

👉 **XOR helps in encryption and decryption.**

Circular Shift ⭐

- Moves bits **left or right**.
- Bits that move out **come back from the other side**.
- Bit values do **not change**.

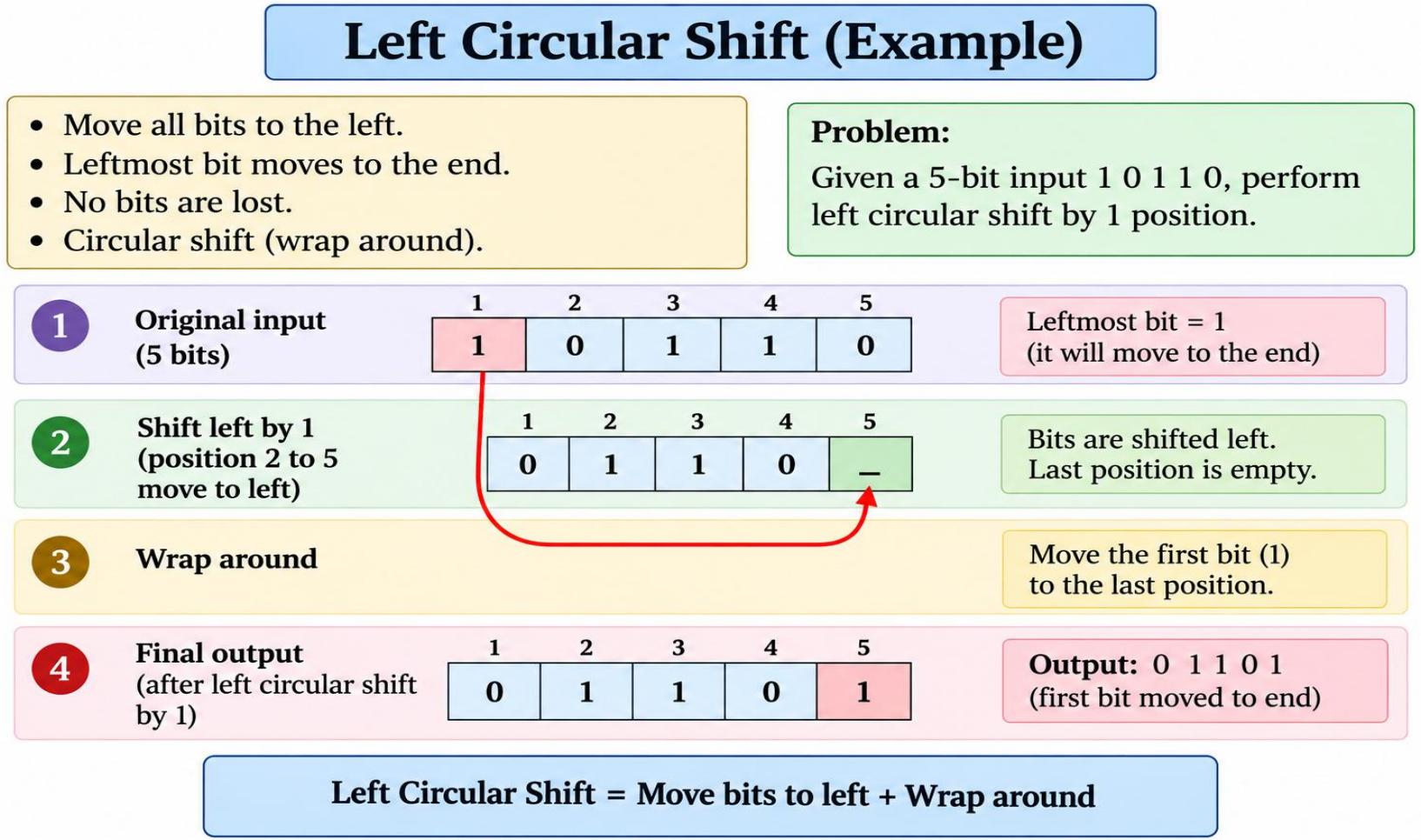
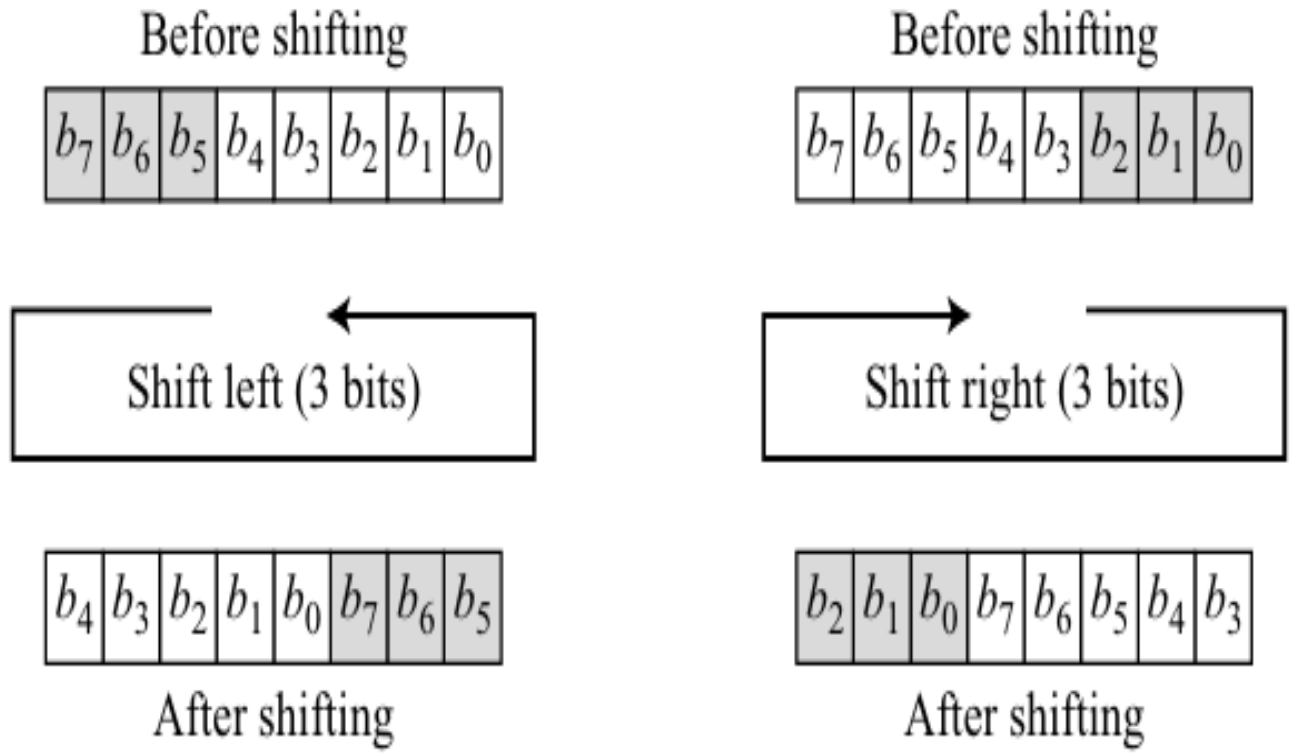
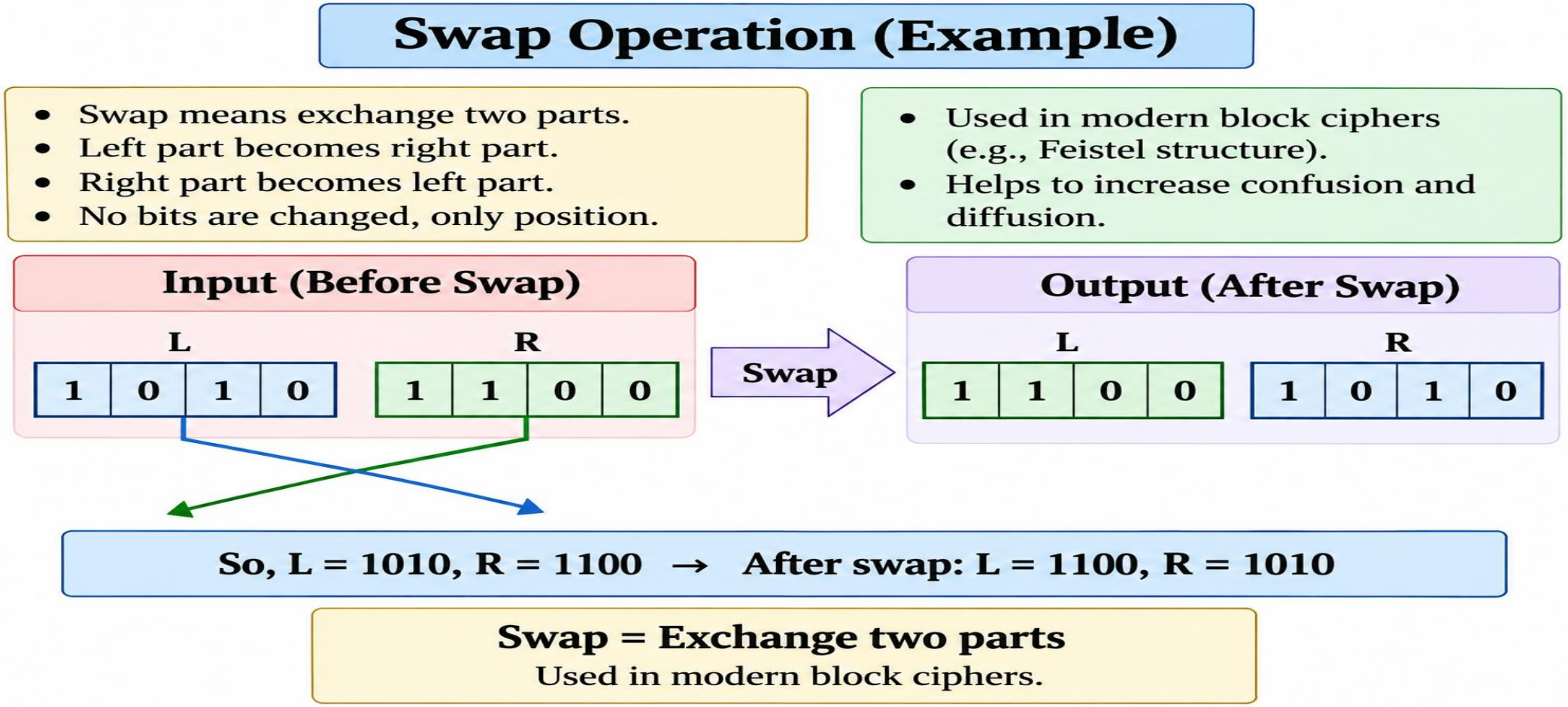


Figure 5.10 *Circular shifting an 8-bit word to the left or right*



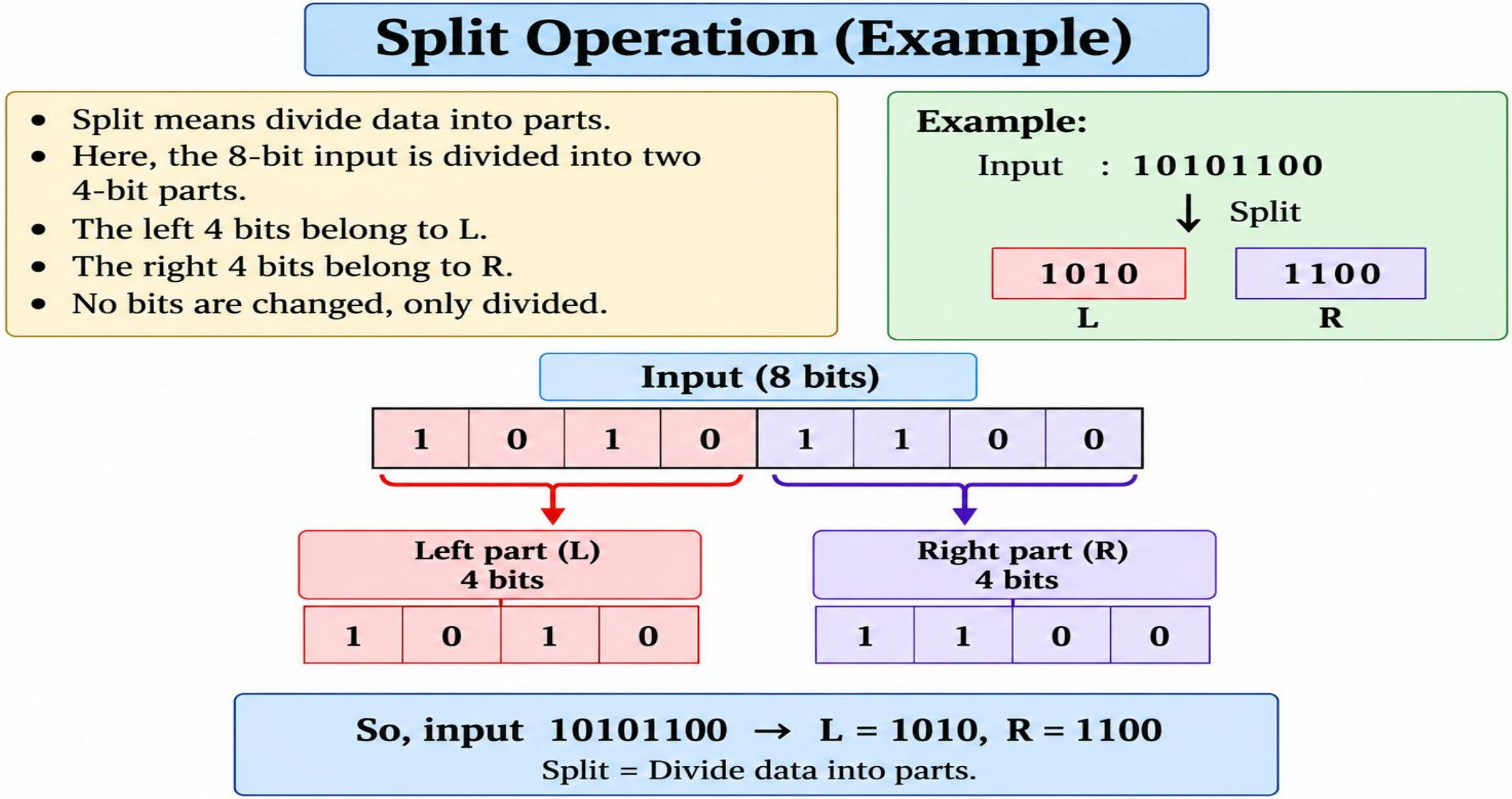
Swap Operation ⭐

- **Swap** means **exchange**.
- It exchanges two parts of the data.
- Usually, a block is divided into **two halves**.
- Then the two halves are exchanged.



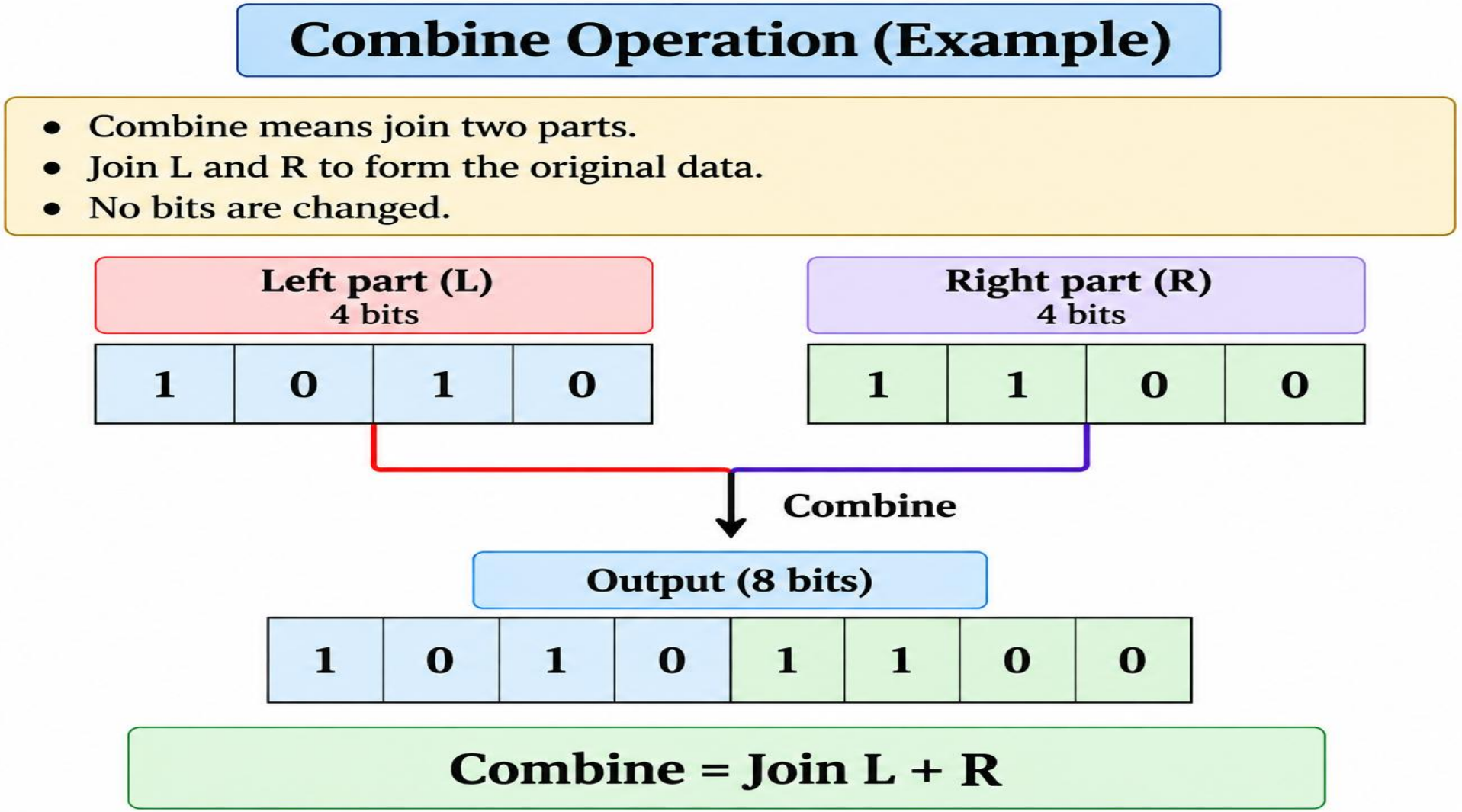
Split Operation ⭐

- **Split** means **divide**.
- It divides a large block into **smaller parts**.
- It is used in modern block ciphers.



Combine Operation

- **Combine = Join two parts**
- Join **L** and **R**.
- No bits are changed.



TWO Classes of Product Ciphers

- A **product cipher** combines different operations such as **S-boxes, P-boxes, and other components**.
- It is used to provide better **security**.
- Modern block ciphers are generally **product ciphers**.
- They are divided into **two classes**:
  - **Feistel Ciphers**
  - **Non-Feistel Ciphers**
- **DES** is an example of a **Feistel cipher**.
- **AES** is an example of a **Non-Feistel cipher**.

1. Feistel Cipher

- Uses **invertible and non-invertible** components.
- Data is divided into **two halves**.
- Uses several **rounds**.
- **DES** is an example.

Note :

Invertible = Reversible (original input can be recovered)

Non-invertible = Irreversible (original input cant be recovered)

Figure 5.17 *Final design of a Feistel cipher with two rounds*

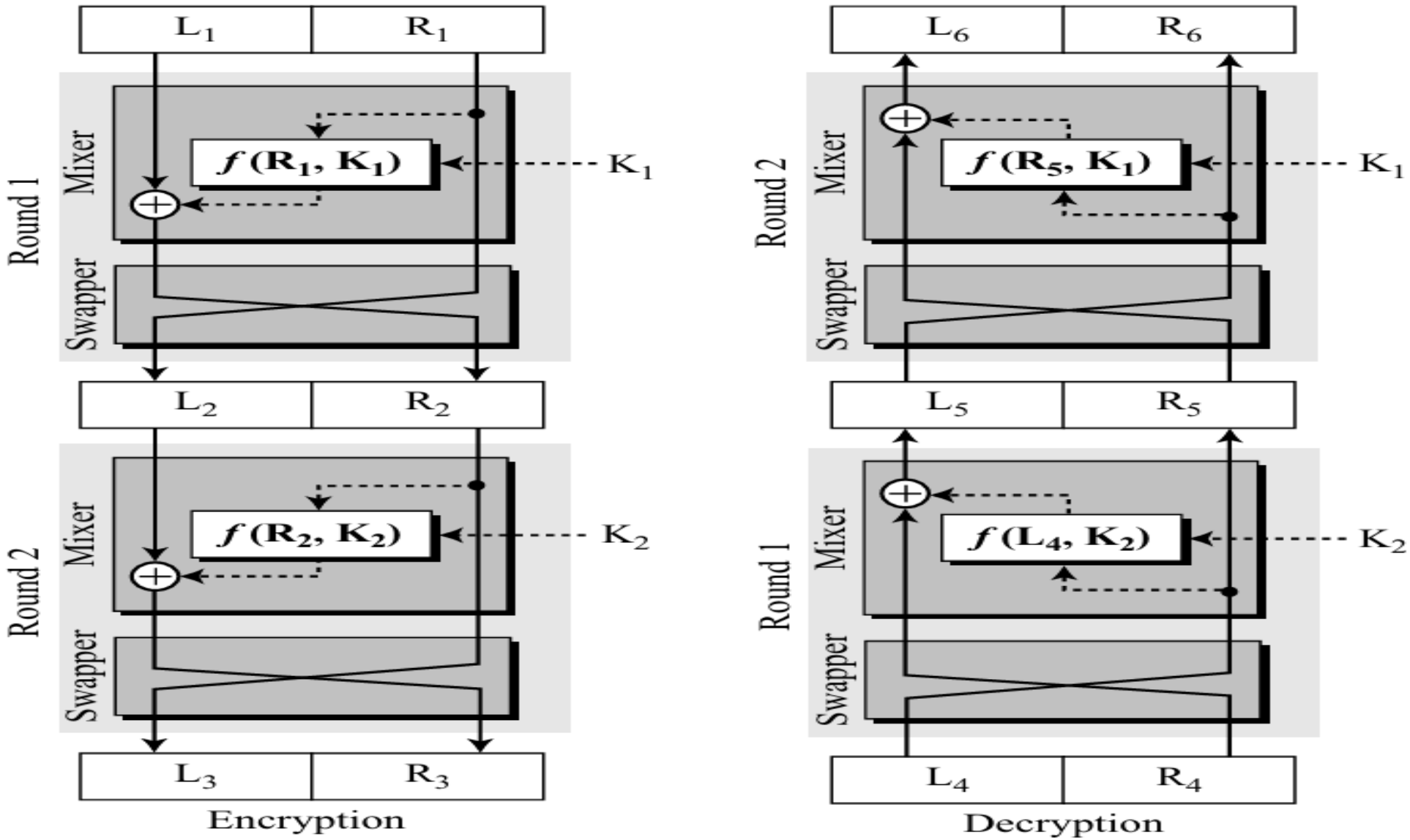


Figure 5.17 – Feistel Cipher with Two Rounds

- **Main Idea**

- **Plaintext** is divided into two halves:
  - L_1 = Left half
  - R_1 = Right half
- Each round uses:
  - **Function f**
  - **Round key K**
  - **XOR ($\oplus$)**
  - **Swapper**

Round 1

1. Input: L_1, R_1
2. Right half R_1 goes to function:
   $f(R_1, K_1)$
3. Output is XORed with L_1:
   $L_1 \oplus f(R_1, K_1)$
4. **Swapper** exchanges the two halves.
5. Output becomes L_2, R_2.

Round 2

1. Input: L_2, R_2
2. Right half R_2 goes to:
   $f(R_2, K_2)$
3. XOR with L_2:
   $L_2 \oplus f(R_2, K_2)$
4. Again, halves are swapped.
5. Final output = L_3, R_3
6. This is the **ciphertext**.

🔴 Decryption — Right Side

Decryption follows the **reverse process**.

Round 1

1. Input: **L_4, R_4**
2. Uses **K_2**.
3. Calculate:
   $f(L_4, K_2)$
4. XOR operation is performed.
5. Swapper exchanges the halves.
6. Output = **L_5, R_5**

Round 2

1. Input: **L_5, R_5**
2. Uses **K_1**.
3. Calculate:
   $f(R_5, K_1)$
4. XOR operation is performed.
5. Output = **L_6, R_6**
6. This gives the **original plaintext**.

Encryption keys:

$K_1 \rightarrow K_2$

Decryption keys:

$K_2 \rightarrow K_1$

⭐ **Easy Flow**

ENCRYPTION

$L_1\ R_1$

↓

Round 1 + K_1

↓

$L_2\ R_2$

↓

Round 2 + K_2

↓

$L_3\ R_3$

DECRYPTION

$L_4\ R_4$

↓

Round 1 + K_2

↓

$L_5\ R_5$

↓

Round 2 + K_1

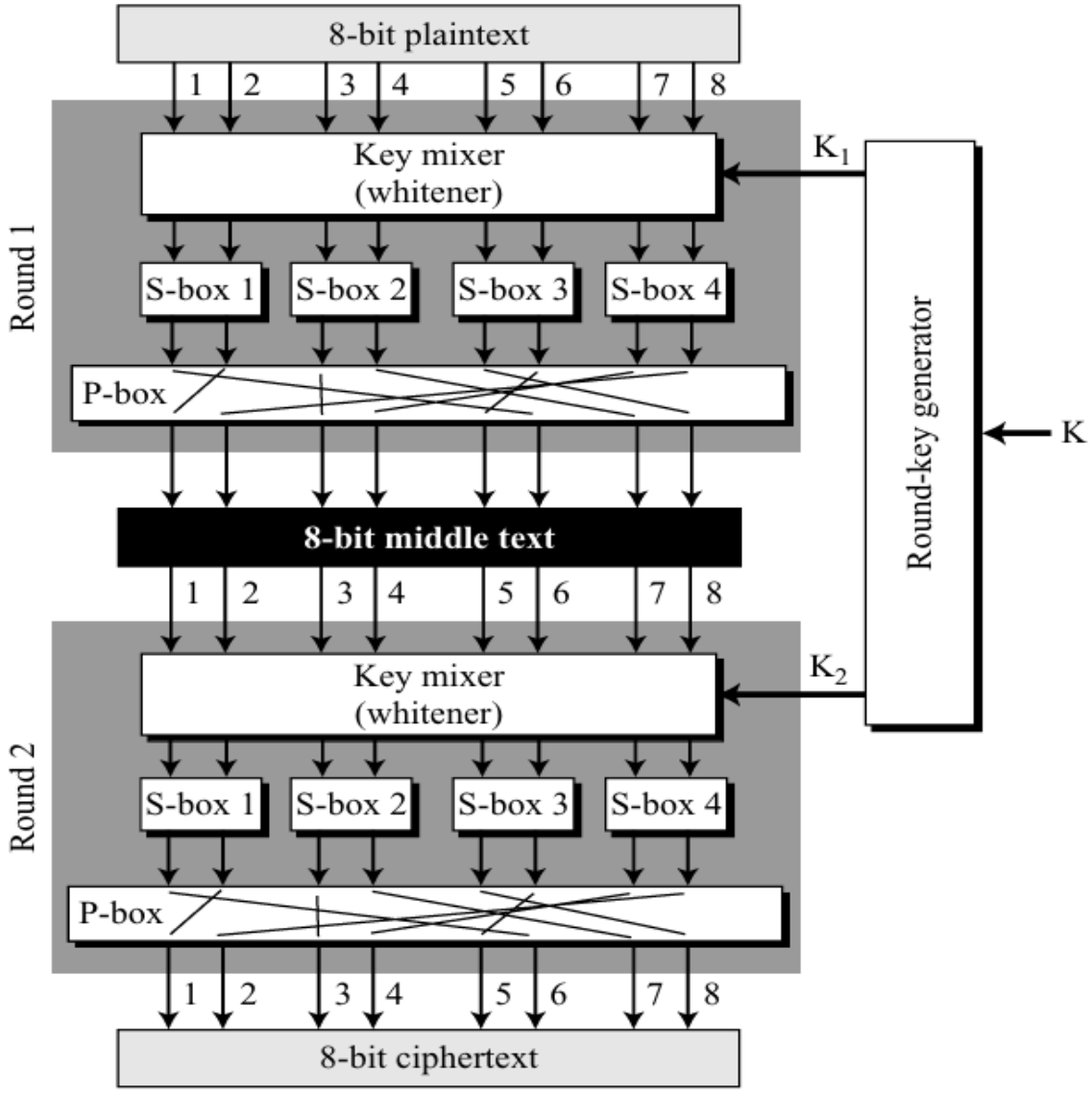
↓

$L_6\ R_6$

2. Non-Feistel Cipher

- Uses **only invertible components**.
- No need to divide data into two halves.
- Components must be **invertible**.
- **AES** is an example.

Figure 5.13 *A product cipher made of two rounds*



Round 1

1. **Key mixer** → mixes plaintext with **K_1**.
2. **S-boxes** → substitute the bits.
3. **P-box** → rearranges the bits.
4. Output → **8-bit middle text**.

Round 2

1. Middle text enters **key mixer**.
2. Uses **K_2**.
3. **S-boxes** substitute the bits again.
4. **P-box** rearranges the bits.
5. Output → **8-bit ciphertext**.

🔑 Keys

K → Round-key generator

↓

K_1, K_2

- **K** = main key
- **K_1** = Round 1 key
- **K_2** = Round 2 key

⭐ Components

Component	meaning
Key mixer	Mixes key + data
S-box	Substitutes bits
P-box	Rearranges bits
Round	One processing stage

⭐ Remember

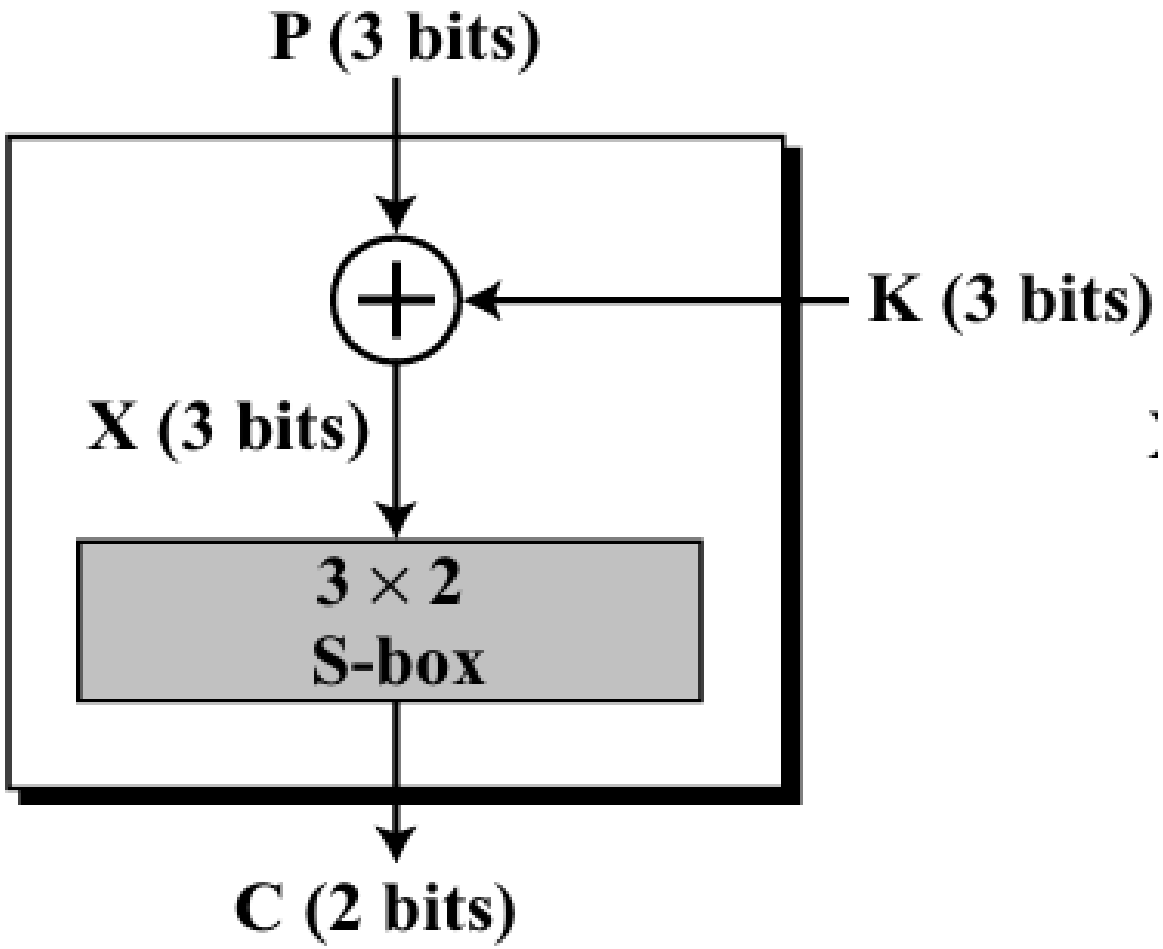
Key Mixer → S-box → P-box → Repeat → Ciphertext

Modern Block Ciphers → **Attacks for Block Ciphers**

1. What is an Attack?

- **Attack** = method to break a cipher.
- Goal → find the **secret key**.
- Modern block ciphers resist many traditional attacks.
- Two important attacks are:
  - **Differential Cryptanalysis**
  - **Linear Cryptanalysis**

- **Differential Cryptanalysis** is an attack used to find the **secret key** of an encryption system.
- In a **Chosen Plaintext Attack**, the attacker can choose the plaintext.
- The attacker then gets the corresponding ciphertext.
- By comparing many plaintext–ciphertext pairs, the attacker tries to find the secret key **K**.



Given S-Box

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

Differential Cryptanalysis

Given

Encryption has two steps: Plaintext → XOR with Key → S-Box → Ciphertext

Given S-Box

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

Step 1:

The secret key **K is unknown**.

The attacker chooses:

- **P_1 = 101**
- **P_2 = 110**

The system gives:

- **C_1 = 11**
- **C_2 = 01**



Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

Goal:
Find the secret key K.

Step 1: Find the Difference Between Plaintexts

We have:

$P_1 = 101$

$P_2 = 110$

Encryption:

$\Delta P = P1 \oplus P2$

For P_1:

```
P1 = 101
p2 = 110
-----------
     011
```

Therefore:

$$\boxed{\Delta P = 011}$$

The two plaintexts differ by 011.

Step 2: Find the Difference Between Ciphertexts

We have:

$C_1 = 11$

$C_2 = 01$

Again use XOR.

```
 11
      ⊕
 01
------
 10
```

Therefore:

$\Delta C = 10$

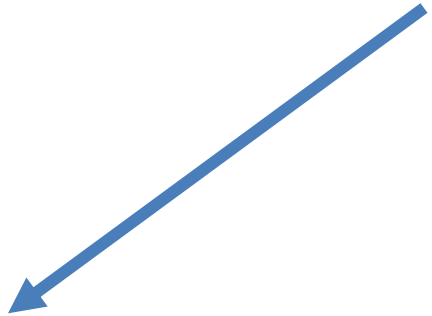
The two ciphertexts differ by 10.

Step 3: Look at the First Ciphertext

We know:

$C_1 = 11$

Now look at the S-Box.
Find all inputs that give output **11**.



Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

Therefore:

X_1 can be 010 (X1 =010) or (X1= 101)
OR
X_1 can be 101 , So we have two possibilities.

Step 4: Find Possible Key – Case 1 (for p1)

We know:

$$X = P \oplus K$$

Therefore:

$$K = P \oplus X$$

We know:

$P_1 = 101$

$X_1 = 010$

So:

$$\begin{array}{r} 101 \\ \oplus\ 010 \\ \hline 111 \end{array}$$

Therefore: K=111

Step 5: Find Possible Key – Case 2

$$X_1 = 101\ ,\ P_1 = 101$$

Again: XOR operation

$$\begin{array}{r} 101 \\ \oplus\ 101 \\ \hline 000 \end{array}$$

Therefore: K=000

Now we have:

Possible Key 1 = 111

Possible Key 2 = 000

We **do not know the key yet**

K=P⊕X , now we know keys, Try with P2

Formula : become $X = P \oplus K$, use P2

Step 6: Check the First Possible Key

Take:

Key 1 = 111

Use the second plaintext:

P_2 = 110

Calculate:

$X2 = P2 \oplus K$

```
  110
⊕ 111
 -- -------
  001
```

$X_2 = 001$

Now go to the S-Box. see x2=001

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

$C_2 = 01$ (we are using p2, so c2)

WE GOT A MATCH IN Table

KEY = 111 works.

Step 7: Check the Second Possible Key

Now try:

Key 2 = 000

Again:

$P_2 = 110$

Calculate:

```
   110
⊕  000
  -----
   110
```

$X_2 = 110$

Collect $X_2 = 110$

Look at S-Box:

110 → 01

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

$C_2 = 01$

So:

K = 000 also works. ✅

We use both keys , we got match in table c2

Step 8: What Happened?

After checking both pairs, we got:

`K = 111 → works`

`K = 000 → works`

So **we cannot find the unique key using only these two plaintext-ciphertext pairs**.

We need more pairs.

For example:

$P_3 \rightarrow C_3$

$P_4 \rightarrow C_4$

$P_5 \rightarrow C_5$

Then we test:

K = 111

K = 000

The wrong key will eventually be eliminated.

Step 9: Why Differential Cryptanalysis Is Useful

The attacker does not simply try every key blindly.
Instead, the attacker studies:

```
Plaintext difference
        ↓
      ΔP
        ↓
    S-Box
        ↓
      ΔC
        ↓
Possible key values
        ↓
Check with more pairs
        ↓
Secret key
```

Differential Cryptanalysis – Steps

1. **Take P_1 and C_1**
    1. Use the S-Box.
    2. Find possible **X1 values**.
2. **Find possible keys**
    1. Use:

$$K = P_1 \oplus X$$

1. **Take P_2**
    1. Try each possible key with P_2.
2. **Calculate X_2**

$$X_2 = P_2 \oplus K$$

1. **Use the S-Box**
    1. Find the output C_2.
2. **Compare with actual C_2**
    1. Same → **Key matches** ✅
    2. Different → **Key rejected** ❌
3. **Repeat**
    1. Check with more plaintext-ciphertext pairs.

1. **Final**
    1. The key that matches all pairs is the **secret key**.

P_1, C_1 → Find Keys → Use Key with P_2 → Find C_2 → Match → Keep Correct Key

Part 2 : Attacker has some more analysis

We use:

- **ΔP = 011**
- S-Box

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

Step 1: Write ΔP

$$\boxed{\Delta P = 011}$$

This means:
P1⊕P2=011

So: shift value; find out p2
P2=P1⊕011

Step 2: Choose P_1

There are 8 possible values.

$P_1 = 000$

$P_1 = 001$

$P_1 = 010$

$P_1 = 011$

$P_1 = 100$

$P_1 = 101$

$P_1 = 110$

$P_1 = 111$

Step 3: Find P_2(try all 8 p1 in equation)

For each P_1:

$$P_2 = P_1 \oplus 011$$

Example 1

$$P_1 = 000$$

```
  000
⊕ 011
  -----
  011
```

So:

$$P_2 = 011$$

Example 2

$$P_1 = 001$$

```
  001
⊕ 011
  -----
  010
```

So:

$$P_2 = 010$$

Do the same for all 8.

Step 4: Look at the S-Box

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

$$P_1 = 000$$

$$P_2 = 011$$

According to p1 and p2
C1 and c2 is

C1 = 00
C2=10
Map all possibility
like c3 ,c4 ,c5 ,c6,c7,c8

Now simply (map) find:

$$P_1 \rightarrow C_1$$

and

P2→C2

Step 5: Make the Table
Now everything comes together:

P1 map c1
P2 map with c2
Finally
C1=00
C2 =10
C1 XOR C2 = 10

Input X	Output C
000	00
001	01
010	11
011	10
100	10
101	11
110	01
111	00

P_1	$P_2 = P_1 \oplus 011$	C_1	C_2	$C_1 \oplus C_2$
000	011	00	10	10

Step 5: LIKEWISE FILL ALL 8 POSSIBILITIES

P_1	$P_2 = P_1 \oplus 011$	C_1	C_2	$C_1 \oplus C_2$
000	011	00	10	10
001	010	01	11	10
010	001	11	01	10
011	000	10	00	10
100	111	10	00	10
101	110	11	01	10
110	101	01	11	10
111	100	00	10	10

Step 6: Count the Last Column

Look only at the last column:

10
10
10
10
10
10
10
10

So: table 5.4

Output difference	Count
00	0
01	0
10	8
11	0

ΔP	00	01	10	11
011	0	0	**8**	0

Step 7: Convert to Probability
Divide by 8.

```
00 → 0/8 = 0
01 → 0/8 = 0
10 → 8/8 = 1
11 → 0/8 = 0
```

ΔP	00	01	10	11
011	0	0	**1.00**	0

⭐ **Remember Only This**

1. Fix ΔP = 011

↓

2. Choose P_1 from 000 to 111

↓

3. Calculate $P_2 = P_1 \oplus 011$

↓

4. Use S-Box to get C_1, C_2

↓

5. Calculate $C_1 \oplus C_2$

↓

6. Count the answers → Table 5.4

↓

7. Divide by 8 → Table 5.5

For our example:

$$\boxed{011 \rightarrow 10}$$

occurs **8 out of 8 times**.

Therefore:

P(ΔC=10|ΔP=011)=1.00=100%

Meaning this table

When the input changes by 011, the output changes by 10

Since it happened **8 out of 8 times**:
Probability = 100%.

Linear Cryptanalysis

- It is a method to **find the secret key**.
- We use **known plaintext and ciphertext**.
- We look for a **simple relationship between their bits**.
- This relationship helps us guess the **key bits**.
- It uses **XOR (⊕)** operations.

Suppose:

$$C_0 = P_0 \oplus K_0 \oplus P_1 \oplus K_1$$

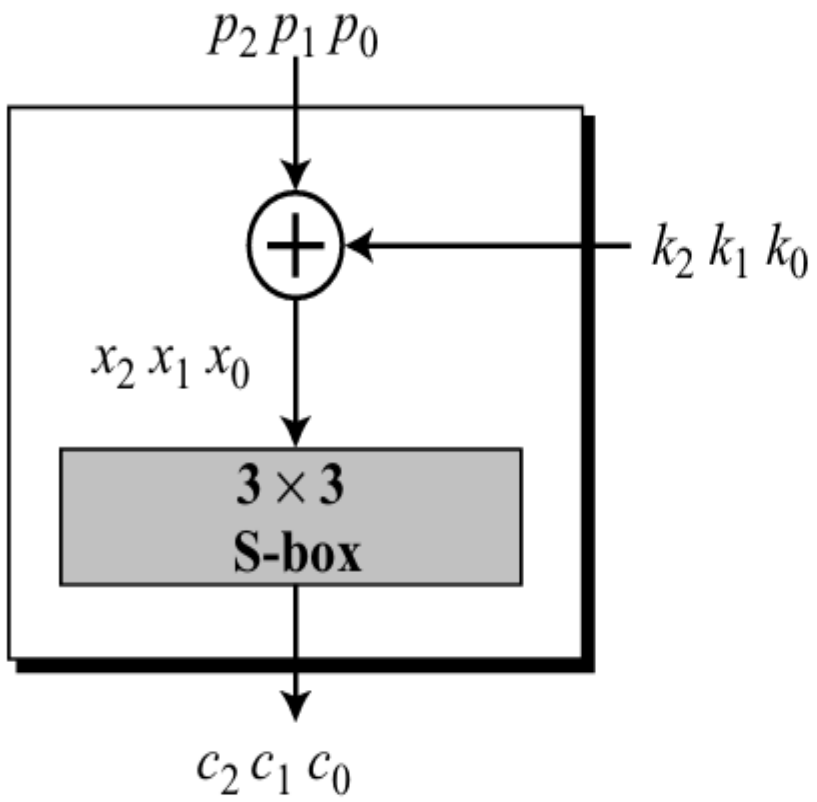
We know:

- $P_0, P_1 \rightarrow$ plaintext
- $C_0 \rightarrow$ ciphertext
- $K_0, K_1 \rightarrow$ unknown key bits

Known Plaintext → Find pattern → Guess key bits

Linear Cryptanalysis

A simple cipher with a linear S-box



$$c_0 = x_0 \oplus x_1$$
$$c_1 = x_0 \oplus x_1 \oplus x_2$$
$$c_2 = x_1 \oplus x_2$$

S-box

Linear Cryptanalysis – Example

1. Plaintext

We start with:

$$P = p_2 p_1 p_0$$

Example:

P = 101

2. Secret Key

Key is:

$$K = k_2 k_1 k_0$$

Example:

K = 011

3. XOR Plaintext and Key

$$X = P \oplus K$$

P= 101

K=011

$$\begin{array}{rr} & 101 \\ \oplus & 011 \\ & \text{-----} \\ X= & 110 \end{array}$$

4. S-Box

The **S-box changes X into C**.

gives these rules:

$$c_0 = x_0 \oplus x_1$$

$$c_1 = x_0 \oplus x_1 \oplus x_2$$

c2=x1⊕x2

For:

X = 110

So:

- $x_2 = 1$
- $x_1 = 1$
- $x_0 = 0$

5. Calculate C

For c_0:

$$c_0 = 0 \oplus 1 = 1$$

For c_1:

$$c_1 = 0 \oplus 1 \oplus 1 = 0$$

For c_2:

$$c_2 = 1 \oplus 1 = 0$$

Therefore:

C=c2 ,c1 and c0 =001

Complete Example

```
Plaintext          = 101
        ↓
Secret Key         = 011
        ↓ XOR
X                  = 110
        ↓
S-Box
        ↓
Ciphertext         = 001
```

Linear cryptanalysis = Find simple XOR patterns in the cipher and use them to guess the secret key.

Data Encryption Standard (DES):

1. DES History
2. DES Structure
3. DES Analysis
4. Multiple DES
5. Security of DES

DES (Data Encryption Standard) – Key Points

- **Type:** Symmetric Block Cipher
- **Also called:** Data Encryption Algorithm
- **Replaced by:** AES Advanced Encryption Standard (in 2001

Sizes

- **Input size:** 64 bits (plaintext)
- **Output size:** 64 bits (ciphertext)

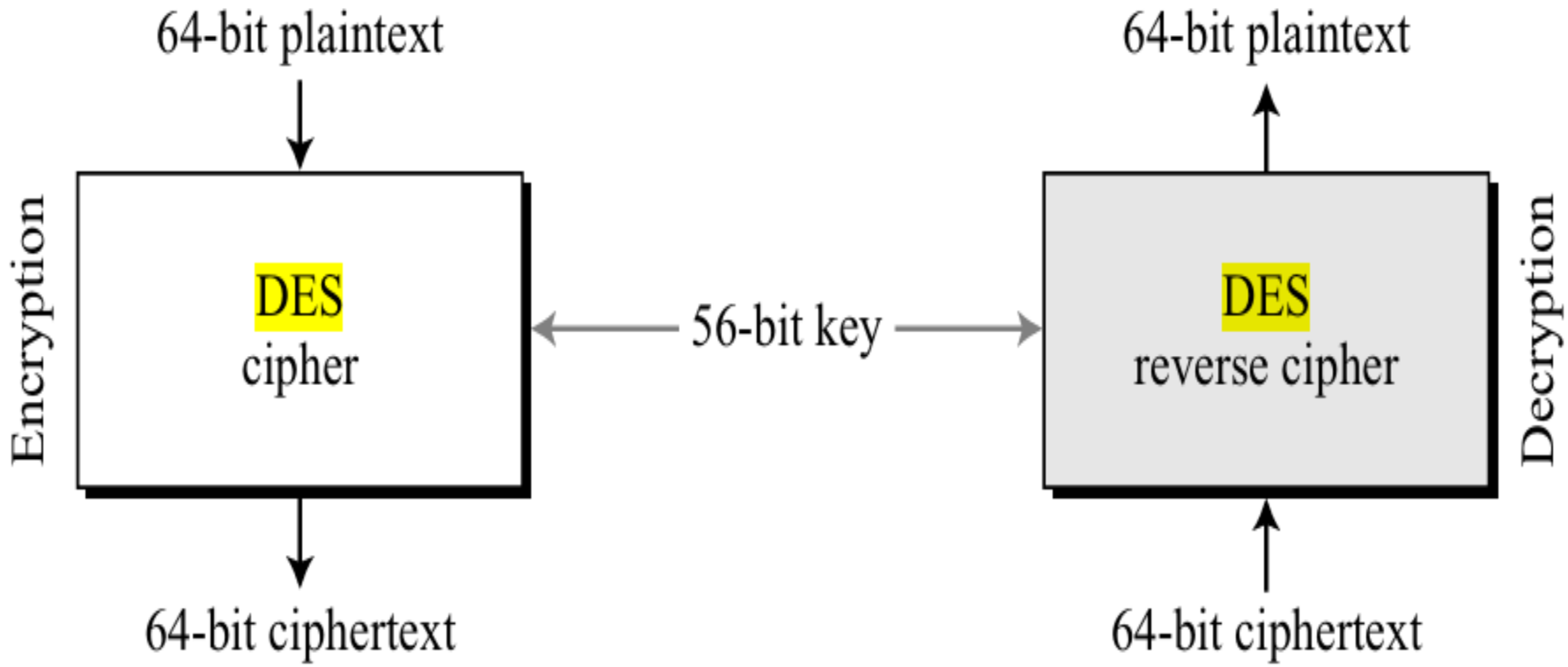
Keys

- **Main key size:** 64 bits
- **Effective key:** 56 bits (8 parity bits removed)
- **Round key size:** 48 bits

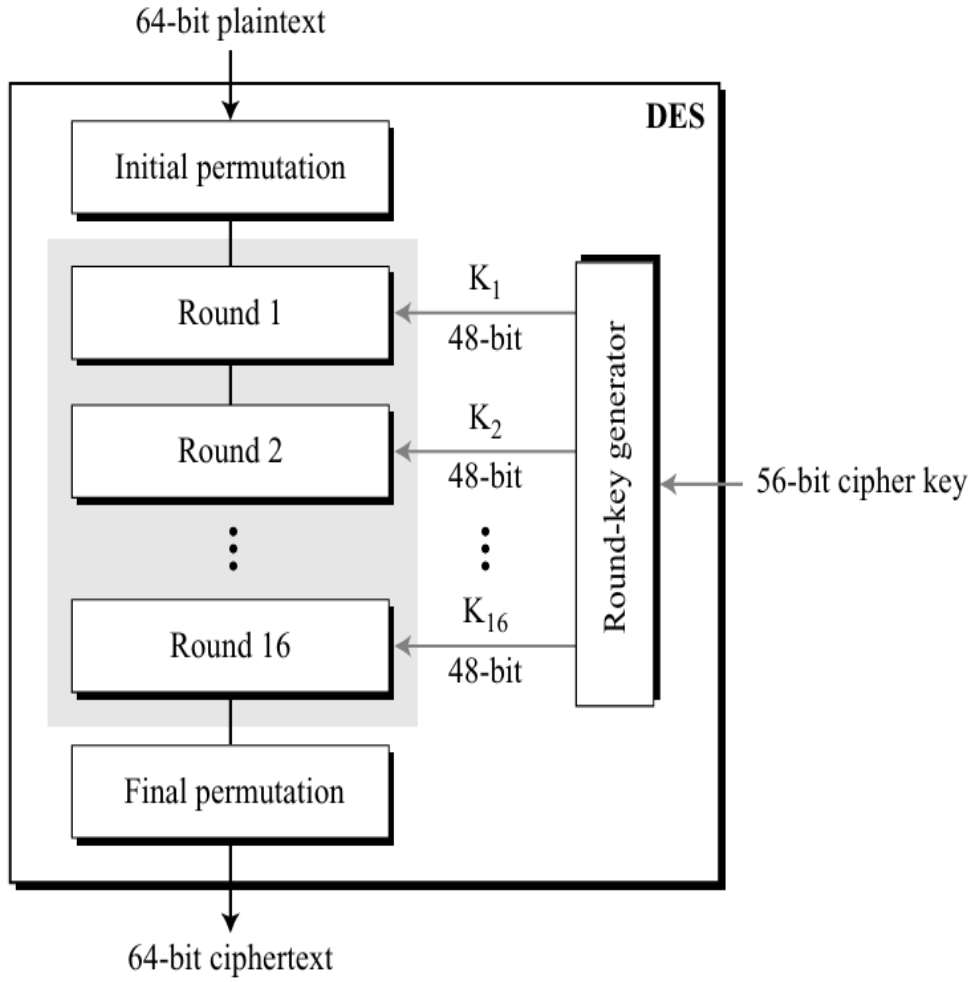
Rounds

- **Number of rounds:** 16 rounds
- **Each round:** uses a different round key 🔐

Figure 6.1 *Encryption and decryption with DES*

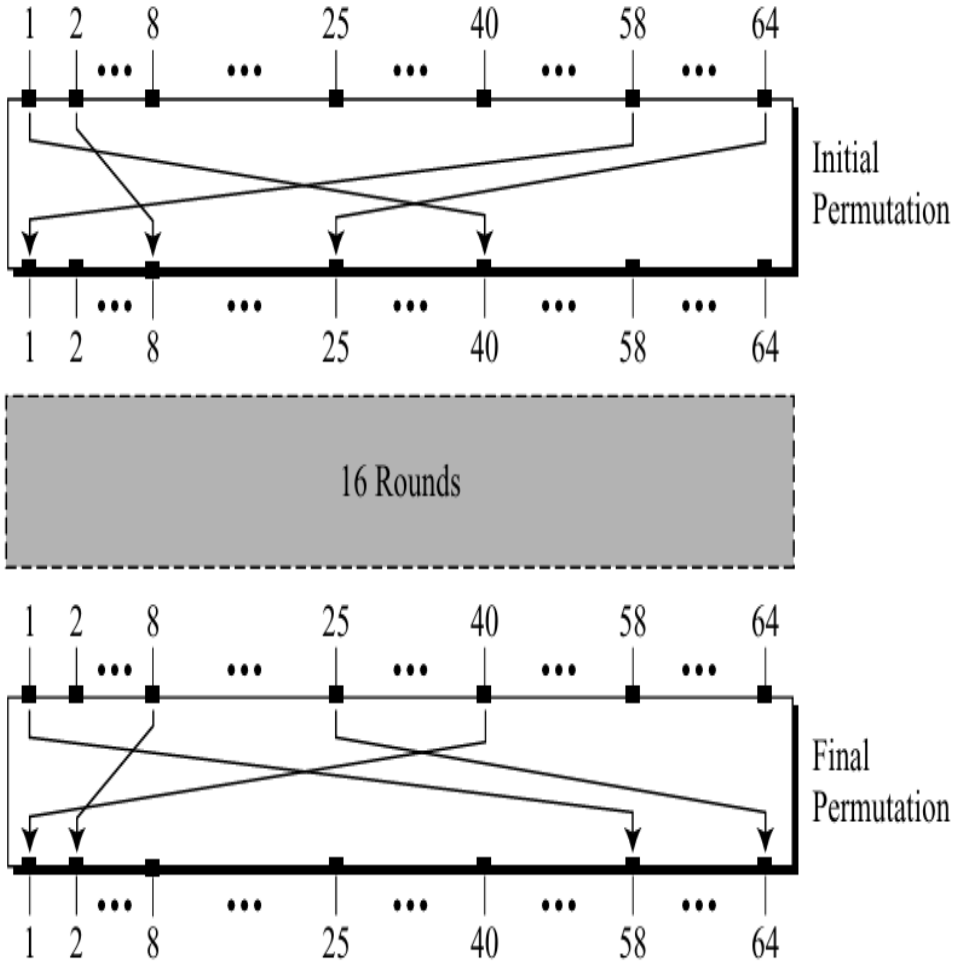


DES STRUCTURE



1. **64-bit Plaintext**
    1. DES takes **64-bit input**.
2. **Initial Permutation**
    1. Rearranges the **64 bits**.
    2. No key is used.
3. **16 Rounds**
    1. Data passes through **16 Feistel rounds**.
    2. Each round uses a different **48-bit round key**.
4. **Round-Key Generator**
    1. Creates **K_1 to K_{16}**.
    2. Keys are generated from the **56-bit cipher key**.
5. **Final Permutation**
    1. Rearranges the bits again.
    2. It is the inverse of the initial permutation.
6. **64-bit Ciphertext**
    1. Final encrypted output is **64 bits**

Figure 6.3 *Initial and final permutation steps in DES*



1. Initial Permutation (IP)

- Takes **64-bit input**.
- **Rearranges the bits**.
- Uses a **fixed rule/table**.
- **No key is used**.
- Then data goes to **16 rounds**.

2. 16 Rounds

- Data passes through **16 rounds**.
- Each round uses a **48-bit round key**.

3. Final Permutation (FP)

- Takes the output of **Round 16**.
- Again **rearranges the 64 bits**.
- Uses a **fixed rule/table**.
- **No key is used**.
- Produces **64-bit ciphertext**.

Important Point

- **FP is the reverse of IP.**
- Example:
  - IP: **58 → 1**
  - FP: **1 → 58**

64-bit Input → IP → 16 Rounds → FP → 64-bit Ciphertext

Table 6.1 *Initial and final permutation tables*

Initial Permutation	*Final Permutation*
58 50 42 34 26 18 10 02	40 08 48 16 56 24 64 32
60 52 44 36 28 20 12 04	39 07 47 15 55 23 63 31
62 54 46 38 30 22 14 06	38 06 46 14 54 22 62 30
64 56 48 40 32 24 16 08	37 05 45 13 53 21 61 29
57 49 41 33 25 17 09 01	36 04 44 12 52 20 60 28
59 51 43 35 27 19 11 03	35 03 43 11 51 19 59 27
61 53 45 37 29 21 13 05	34 02 42 10 50 18 58 26
63 55 47 39 31 23 15 07	33 01 41 09 49 17 57 25

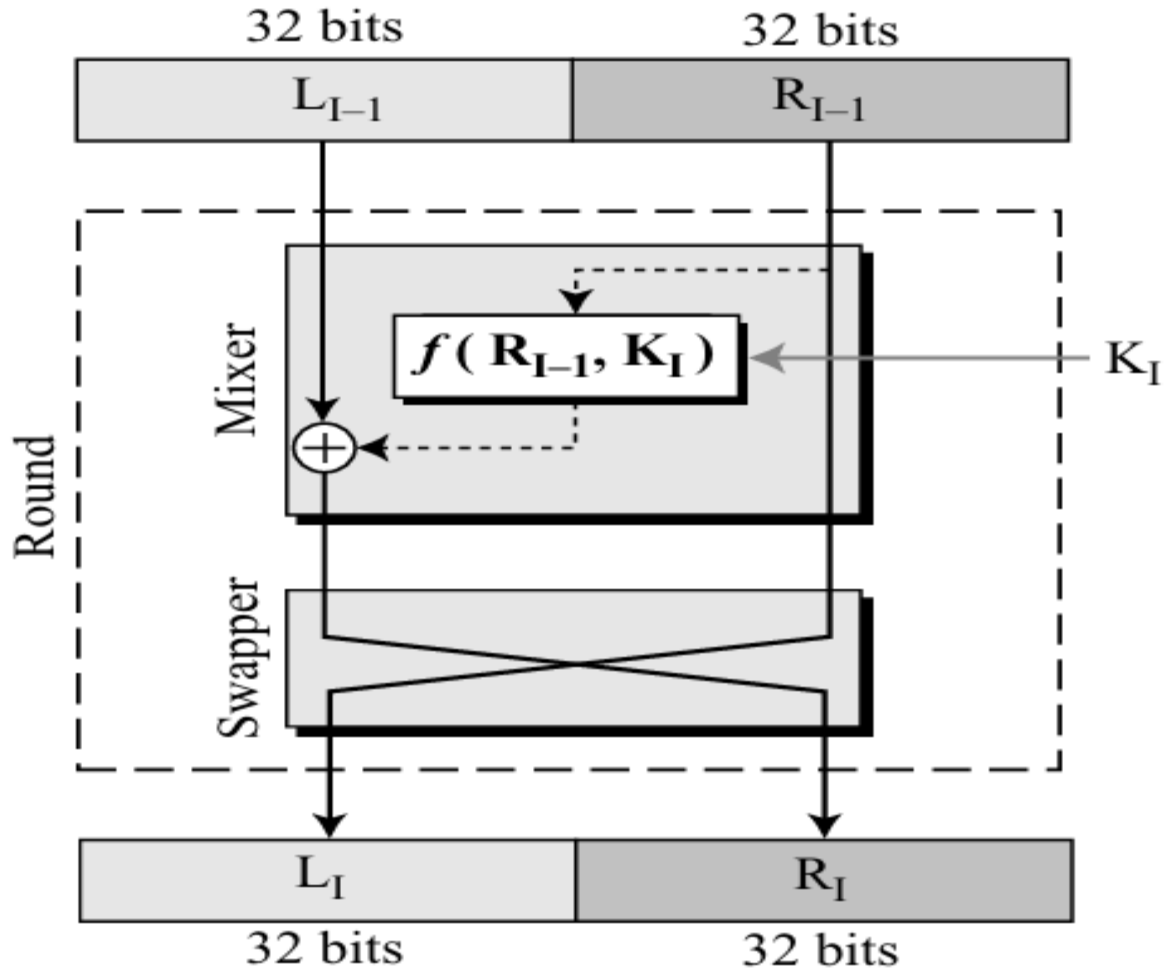
DES Initial Permutation (IP) and Final Permutation (FP) Table.

1. Initial Permutation Table
Left side = Initial Permutation
It tells DES where each input bit should go.
DES takes 64 bits and rearranges them.
Example:
Input bit 58 → Output bit 1
Input bit 50 → Output bit 2
Input bit 42 → Output bit 3

2. Final Permutation Table
Right side = Final Permutation
It tells DES how to rearrange the bits after 16 rounds.
Example:
Input bit 40 → Output bit 1
Input bit 08 → Output bit 2
Input bit 48 → Output bit 3

3. Important Point
Both tables contain 64 positions.
No key is used.
They only rearrange the bits.
Final Permutation is the inverse of Initial Permutation.

Figure 6.4 *A round in DES (encryption site)*



1. **Split the Data**

- After IP, **64 bits** are divided.
- **L_0 = 32 bits**
- **R_0 = 32 bits**

2. 16 Rounds

- DES has **16 Feistel rounds**.
- Each round uses one **48-bit key**.
- Keys: **K_1 to K_{16}**

3. Each Round

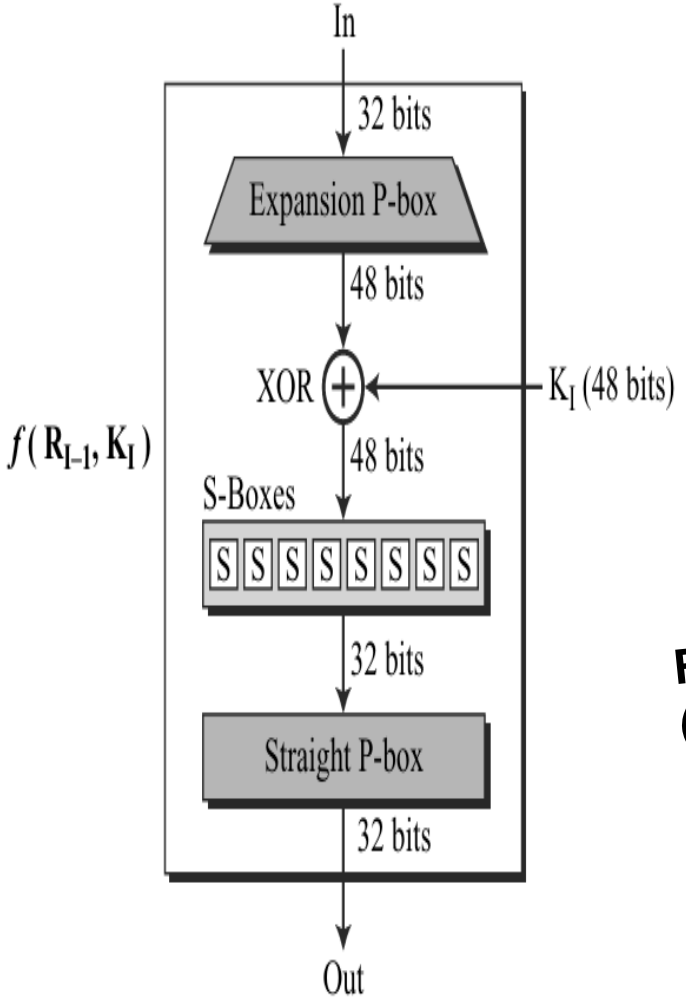
- Right part → **Expansion**
- XOR with **round key**
- Pass through **S-Boxes**
- Pass through **P-Box**
- XOR with **left part**
- Parts are exchanged for the next round.

4. Formula

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

Figure 6.5 *DES function*



DES Round Function

1. **Take R** – Right side has **32 bits**.
2. **Expand** – 32 bits become **48 bits**.
3. **XOR Key** – Mix with the **48-bit round key**.
4. **S-Boxes** – Change **48 bits → 32 bits**.
5. **P-Box** – Rearranges the **32 bits**.
6. **XOR with L** – Mix with the **left 32 bits**.
7. **New R** – This gives the new right side.
8. **New L** – Old right side becomes the new left side.

R (32) → Expansion (48) → XOR K → S-Boxes (32) → P-Box (32) → XOR L → New R
New L = Old R

DES Round — Mixer and Swapper

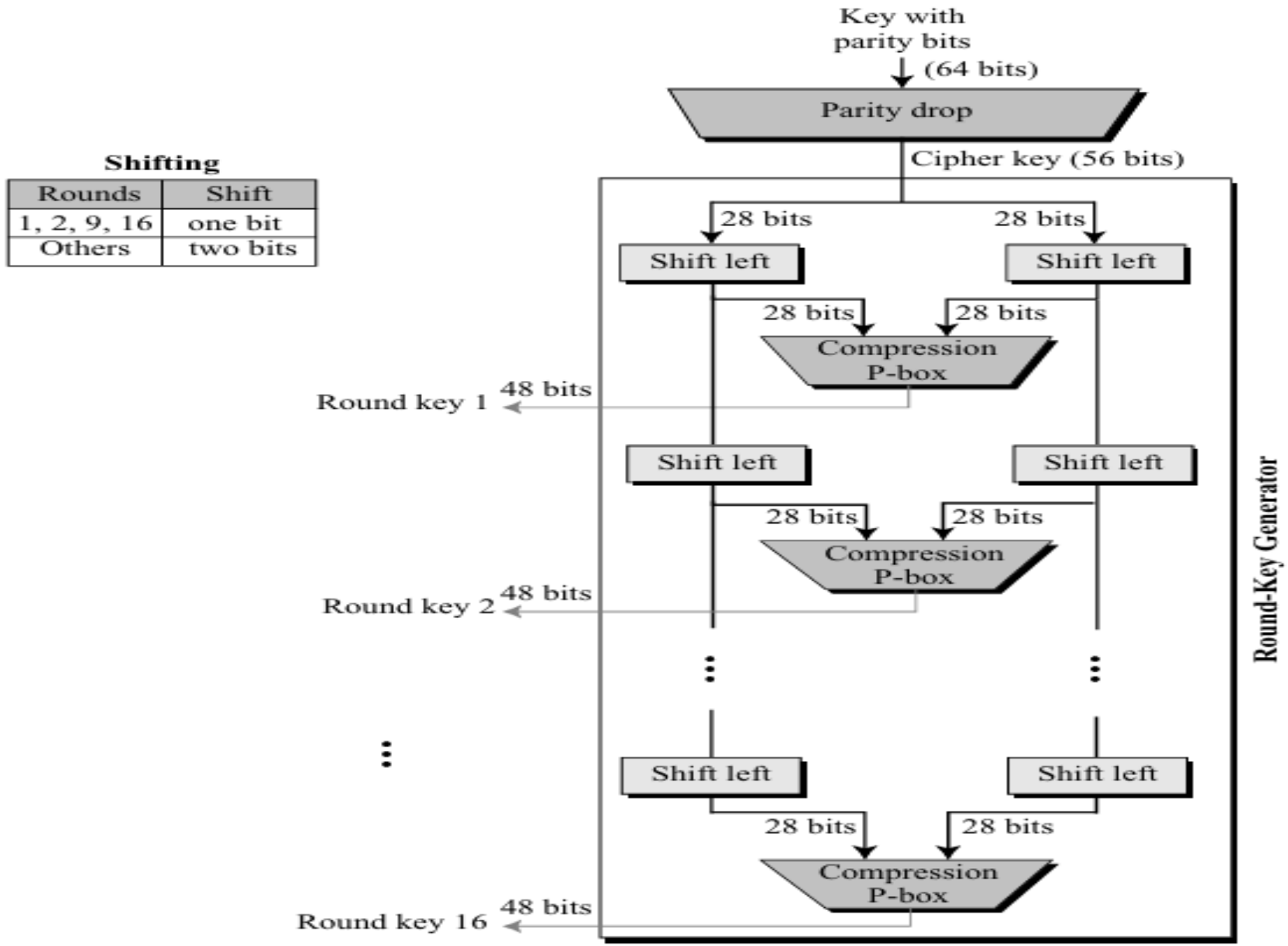
Mixer

- Takes **L and R**.
- Processes the **R part**.
- Uses the **round key K_i**.
- XOR gives the **new R**.

Swapper

- Exchanges **L and R**.
- Old **R** → **New L**.
- New R comes from the **mixer**.

Figure 6.10 *Key generation*



DES Key Generation

- **Purpose:** Generate **16 round keys**.
- Original key = **64 bits**.
- **8 parity bits** are removed.
- Remaining key = **56 bits**.
- Divide 56 bits into two parts:
  - **C = 28 bits**
  - **D = 28 bits**
- Perform **left circular shift** on C and D.
- Combine C and D → **56 bits**.
- Apply **compression permutation**.
- 56 bits → **48 bits**.
- This gives one **round key**.
- Repeat for all 16 rounds.
- Final keys: **K_1, K_2, K_3, …, K_{16}**.
- Each round key = **48 bits**

Example 6.5 — DES Encryption

Given

- **Plaintext:** `123456ABCD132536`
- **Key:** `AABB09182736CCDD`
- Values are in **hexadecimal**.
- 1 hexadecimal digit = **4 bits**.
- Therefore:
  - 16 hex digits × 4 = **64 bits**

Step 1: Take the Plaintext

Plaintext:

`123456ABCD132536`

It contains **16 hexadecimal digits**.

So:

16 × 4 = 64 bits

DES accepts a **64-bit plaintext block**.

Step 2: Apply Initial Permutation

The 64-bit plaintext is given to the **Initial Permutation (IP)**.

What happens?

- Bits are **rearranged**.
- No bits are added.
- No bits are removed.
- Still **64 bits**.

Result from the book:

After Initial Permutation:

`14A7D67818CA18AD`

Step 3: Split into Two Parts

Now divide the 64-bit result into **two equal parts**.

`14A7D678 | 18CA18AD`

Each part has:

8 hexadecimal digits × 4 = 32 bits

Therefore:

- **L_0 = 14A7D678**
- **R_0 = 18CA18AD**

Step 4: Round 1

DES now starts the first round.

Round 1

Input:

- L_0 = `14A7D678`
- R_0 = `18CA18AD`
- K_1 = `194CD072DE8C`

The **mixer** processes the right part using K_1.

Then the **swapper** exchanges the two parts.

Result:

- **L_1 = 18CA18AD**
- **R_1 = 5A78E394**

Step 5: Round 2

Input:

- L_1 = `18CA18AD`
- R_1 = `5A78E394`
- K_2 = `4568581ABCCE`

After mixer and swapper:

- **L_2 = 5A78E394**
- **R_2 = 4A1210F6**

Step 6: Round 3

Input:

- L_2 = `5A78E394`
- R_2 = `4A1210F6`
- K_3 = `6EDA4ACF5B5...` *(see book table for the complete 12-hex-digit key)*

After mixer and swapper:

- **L_3 = 4A1210F6**
- **R_3 = B8089591**

Step 7: Round 4

After Round 4:

- **L_4 = B8089591**
- **R_4 = 236779C2**

Step 8: Continue Rounds

The same process is repeated.

Each round:

1. Take L and R.
2. Use the **right part** in the mixer.
3. Use the **round key**.
4. Produce a new right part.
5. Swap the parts.
6. Go to the next round.

Example 6.5 — DES Encryption (Full Steps)

All values are in Hexadecimal

Given:
Plaintext = 123456ABCD132536 (64 bits)
Key = AABB09182736CCDD (64 bits)

1 Initial Permutation (IP)

- Rearrange the 64 bits according to the Initial Permutation table (given in book).

After IP : 14A7D67818CA18AD (64 bits)

2 Split into two 32-bit halves

- Split the 64-bit result into Left (L_0) and Right (R_0).

14A7D678 | 18CA18AD

L_0 (left 32 bits) R_0 (right 32 bits)

L_0 = 14A7D678
R_0 = 18CA18AD

How split?
64 bits = 16 hex digits
Left 8 digits → L_0
Right 8 digits → R_0

3 Key Generation (Round Keys)

- From the 64-bit key, 16 round keys (K_1 to K_{16}) of 48 bits are generated.
- (Process: parity drop → split → left shift → compression permutation)
- The round keys used in this example are given in the table below.

4 16 Rounds of DES

- For each round i (1 to 16):
- R_i is processed using the round key K_i (through expansion, XOR, S-boxes, permutation).
- Then XOR with L_i and swap the halves (except after round 16).
- The table below shows the Left and Right values after each round.

Round	Left (L_i)	Right (R_i)	Round Key (K_i)
1	18CA18AD	5A78E394	194CD072DE8C
2	5A78E394	4A1210F6	4568581ABCCE
3	4A1210F6	B8089591	06EDA4ACF5B5
4	B8089591	236779C2	DA2D032B6EE3
5	236779C2	A15A4B87	69A629FEC913
6	A15A4B87	2B8F9C65	C1948E87475E
7	2B8F9C65	A9FC20A3	708AD2DB83C0
8	A9FC20A3	308BEB97	34F822F0C66D

Round	Left (L_i)	Right (R_i)	Round Key (K_i)
9	308BEB97	10AF9D37	84B84473DC9C
10	10AF9D37	6CA6CB20	02965708B5BF
11	6CA6CB20	FF3C485F	6D5560AF7CA5
12	FF3C485F	22A5963B	C2C1B96A8BF3
13	22A5963B	387CCDAA	99C31397C91F
14	387CCDAA	BD2DD2AB	251B8BC717D0
15	BD2DD2AB	CF26B472	3330C5D9A36D
16	19BA9212	CF26B472	181C5D75C66D

(No swap after Round 16)

5 Combine after 16 Rounds

- After round 16, combine L_{16} and R_{16} (no swap).

L_{16} = 19BA9212
R_{16} = CF26B472

Combine → 19BA9212CF26B472 (64 bits)

6 Final Permutation (FP)

- Apply the Final Permutation table (inverse of IP). →

Ciphertext = C0B7A8D05F3A829C (64 bits)

Summary:

- Plaintext : 123456ABCD132536
- Key : AABB09182736CCDD
- After IP : 14A7D67818CA18AD
- L_0 : 14A7D678 R_0 : 18CA18AD
- After 16 rounds : L_{16} = 19BA9212, R_{16} = CF26B472
- After combination : 19BA9212CF26B472
- Ciphertext : C0B7A8D05F3A829C

DES Analysis

The main points are:

1.Avalanche Effect

2.Completeness

3.DES Design Criteria

4.Weaknesses of DES

- **Avalanche effect** :
  - A **small change in input**
  - Causes a **large change in output**.

In DES

- Change **one bit** of plaintext.
- Keep the **key the same**.
- Encrypt both plaintexts.
- Compare the two ciphertexts.
- Many ciphertext bits should change.

Example 6.7

Plaintext 1:

0000000000000000

Plaintext 2:

0000000000000001

- Only **1 bit** is changed.
- Same key is used:
  22234512987ABB23

Ciphertexts

For Plaintext 1:

4789FD476E82A5F1

For Plaintext 2:

0A4ED5C15A63FEA3

- Input changed by only **1 bit**.
- Ciphertexts differ in **29 bits**.
- About **1.5%** of plaintext changed.
- About **45%** of ciphertext changed.

Small input change → Large output change

Ciphertext 1: 4789FD476E82A5F1
Ciphertext 2: 0A4ED5C15A63FEA3
Each ciphertext = **64 bits**.

8 4 2 1 + 8 4 2 1
1 0 0 0 + 1 0 0 1
= 8 9.

Step 1: Write in Binary

Ciphertext 1

01000111 10001001 11111101 01000111 01101110 10000010 10100101 11110001

Ciphertext 2

00001010 01001110 11010101 11000001 01011010 01100011 11111110 10100011

Red colour indicates c1 and c2 bit is different

Step 2: Compare Bit by Bit

Bit position	1	2	3	4	5	6	7	8
C1	0	1	0	0	0	1	1	1
C2	0	0	0	0	1	0	1	0

2, 5, 6, 8, 9, 10, 14, 15, 16, 19, 21, 25, 30, 31, 35, 36, 38, 41, 42, 43, 48, 50, 52, 53, 55, 56, 58, 60, 63

Total

- Total bits = **64**
- Changed bits = **29**
- Unchanged bits = **35**

$$64 - 29 = 35$$

Percentage Changed

$$\frac{29}{64} \times 100 = 45.31\%$$

So, **29 out of 64 bits changed = approximately 45%**.

Only **1 bit** was changed in the plaintext, but **29 bits** changed in the ciphertext. This demonstrates the **Avalanche Effect**.

Completeness

- **Completeness** is a property of DES.
- It means **one input bit should affect many output bits**.
- If **one plaintext bit changes**, its effect should spread to many ciphertext bits.
- This helps make the cipher **more secure**.
- It prevents an attacker from easily finding a simple relationship between input and output.

Example

```
1 bit changes in plaintext
          ↓
      DES encryption
          ↓
Many bits change in ciphertext
```

👉 **One input bit → affects many output bits.**

NEXT TOPIC :

Multiple DES

It includes:

- **Double DES**
- **Triple DES (3DES)**

Multiple Encryption and Triple DES

1. Why DES Needs Improvement

- DES uses a **56-bit key**.
- A small key makes **Brute-Force Attack** easier.
- Attacker tries many possible keys.
- So, security of DES needed to be increased.
- **Multiple encryption** uses DES more than once.
- It increases security without changing the basic DES algorithm.

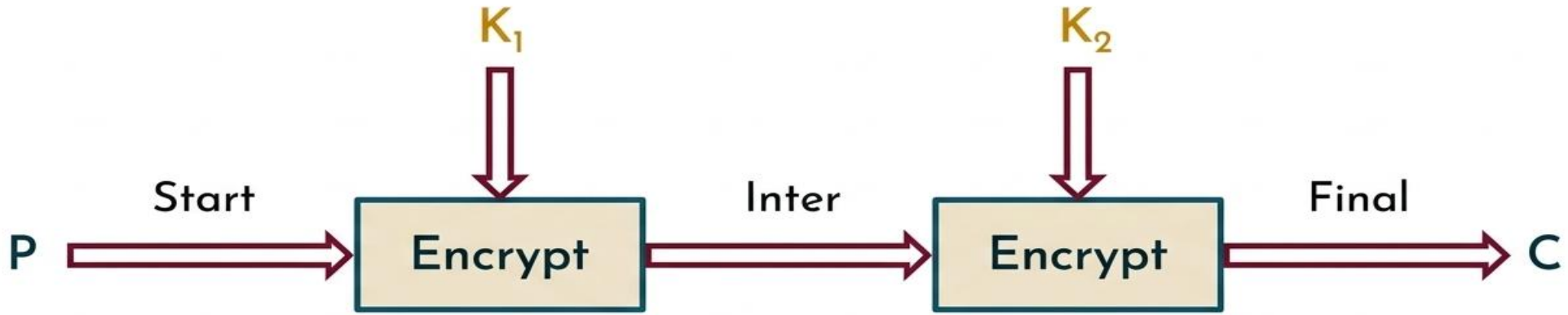
2. Double DES (2DES)

- DES is applied **two times**.
- Two different keys are used:
  - **K_1 = 56 bits**
  - **K_2 = 56 bits**
- Total key size = **112 bits**.

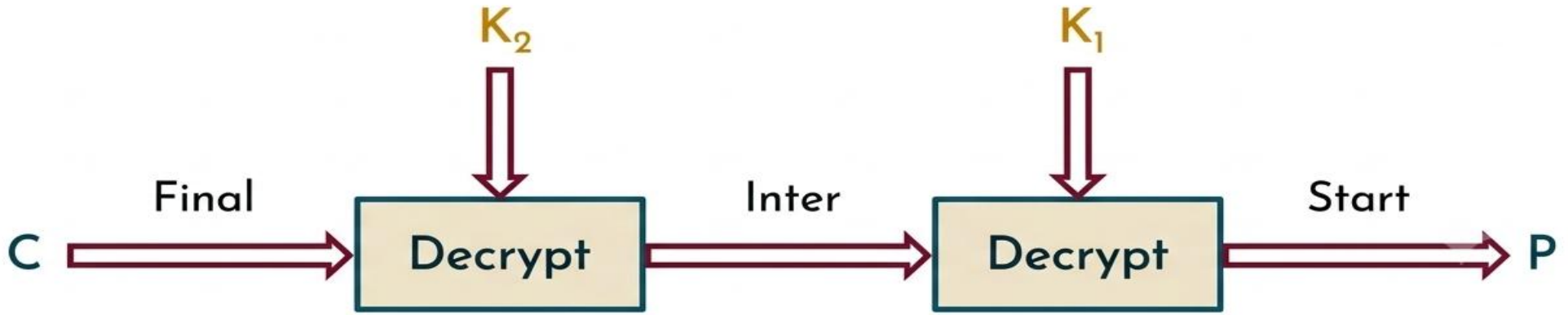
- Encrypt using the same algorithm two times
- And key 1 not equal to key 2

Double DES

Encryption



Decryption



Encryption

Plaintext(start)

↓

DES with K_1

↓

Middle Text(inter)

↓

DES with K_2

↓

Ciphertext(Final)

Decryption

Ciphertext(Final)

↓

DES Decryption with K_2

↓

Middle Text(Inter)

↓

DES Decryption with K_1

↓

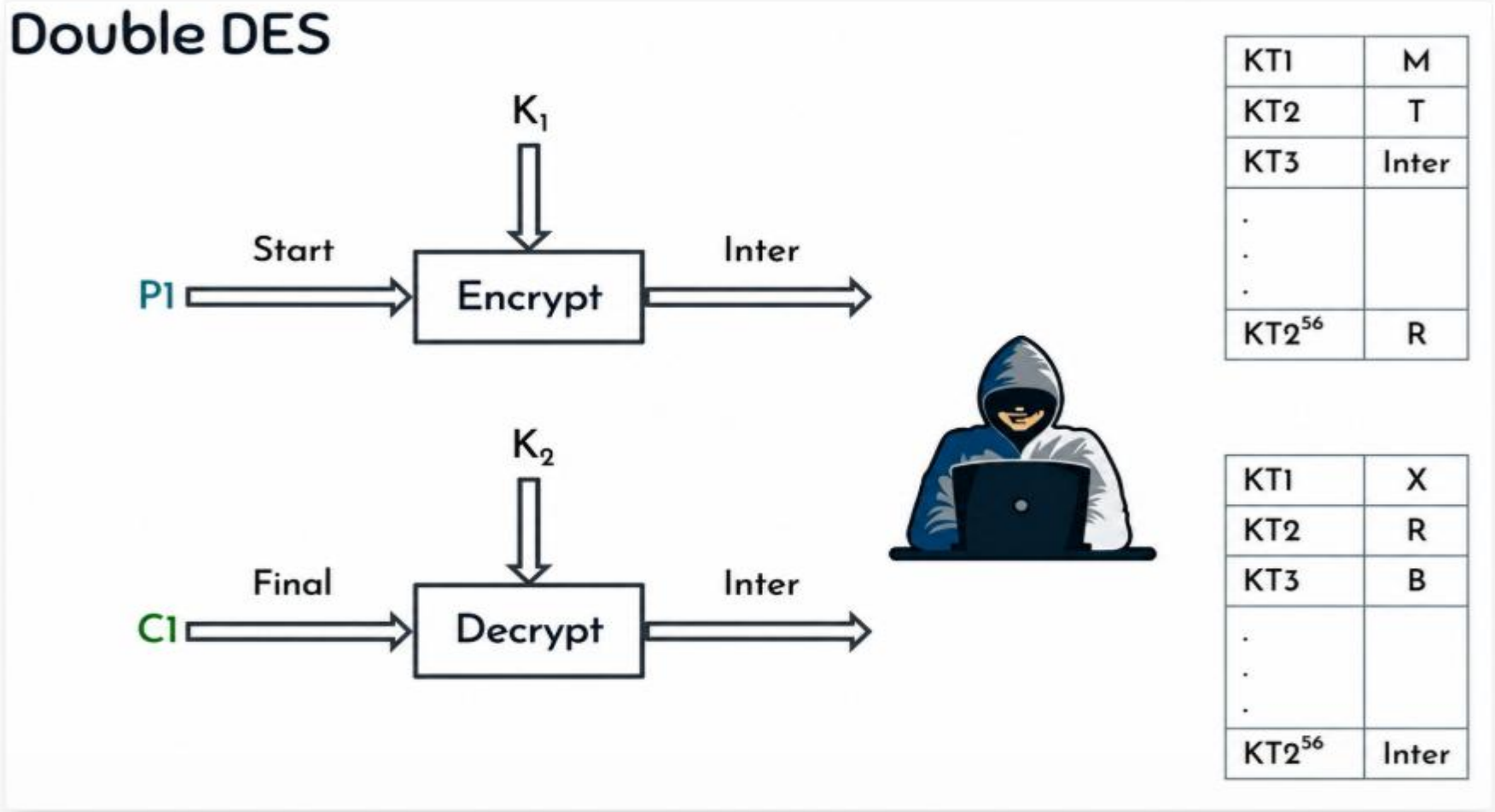
Plaintext(Start)

Key 1 is responsible for generating middle text and Key 2 is responsible for generating ciphertext

Problem with Double DES

- Double DES looks stronger because it uses **112 bits**.
- But it has a weakness called **Meet-in-the-Middle Attack**.

- Keytry2 ➔ second intermediate result



Meet-in-the-Middle Attack – Steps

1.Hacker knows only:

P1 = Plaintext

C1 = Ciphertext

2.First table – Encryption:

- Hacker gives P1 to DES.
- The key is unknown.
- Hacker's goal is to find K_1.
- He tries Key 1, Key 2, Key 3, ... one by one.
- For each trial key, he gets an Intermediate value.
- He stores the key and intermediate values in a database.
- He continues this up to 2^{56} possible keys.

3.Second table – Decryption:

- Hacker takes the known C1.
- He gives C1 to DES decryption.
- He tries possible keys one by one.
- For each key, he gets an Intermediate value.
- He stores these values in another table.

4.Compare both tables:

Hacker compares the Intermediate values from both tables.

When a matching Intermediate value is found, it means the two keys may be correct.

5.Find K_1:

From the first table, the matching Intermediate value gives the corresponding K_1.

6.Find K_2:

From the second table, the same matching Intermediate value gives the corresponding K_2.

Final result:

Hacker gets K_1 and K_2.

These two keys can then be used to decrypt the message.

Simple idea:

P1 → Encryption with K_1 → Intermediate ← Decryption with K_2 ← C1

👉 Match the Intermediate value → Find K_1 and K_2.

Figure 6.14 *Meet-in-the-middle attack for double DES*

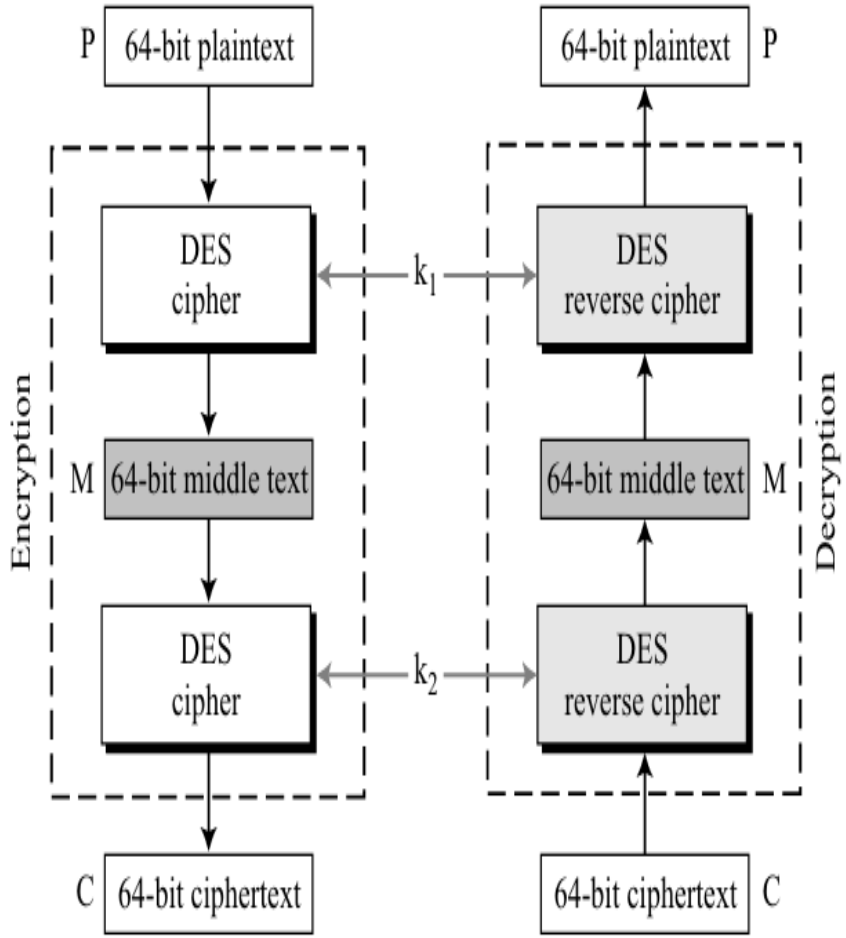
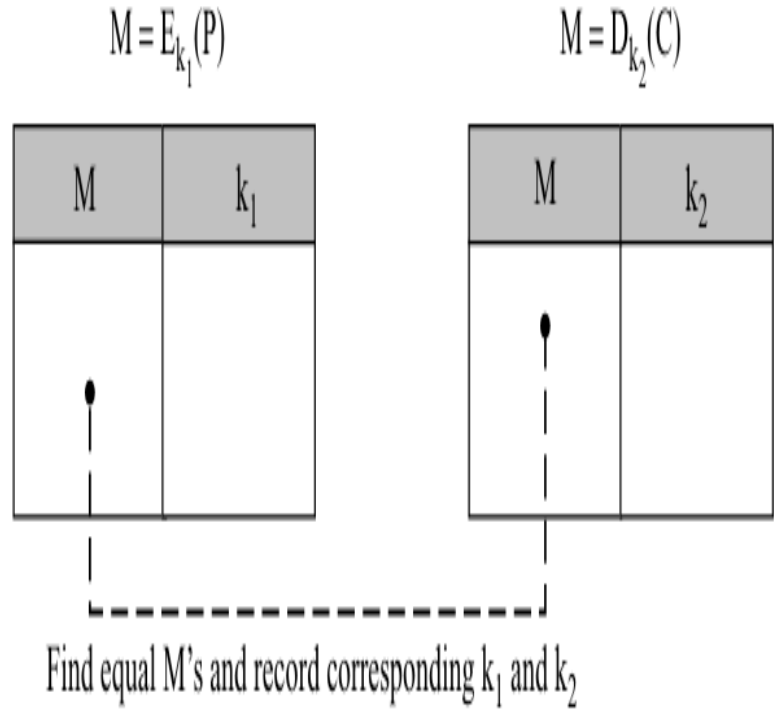
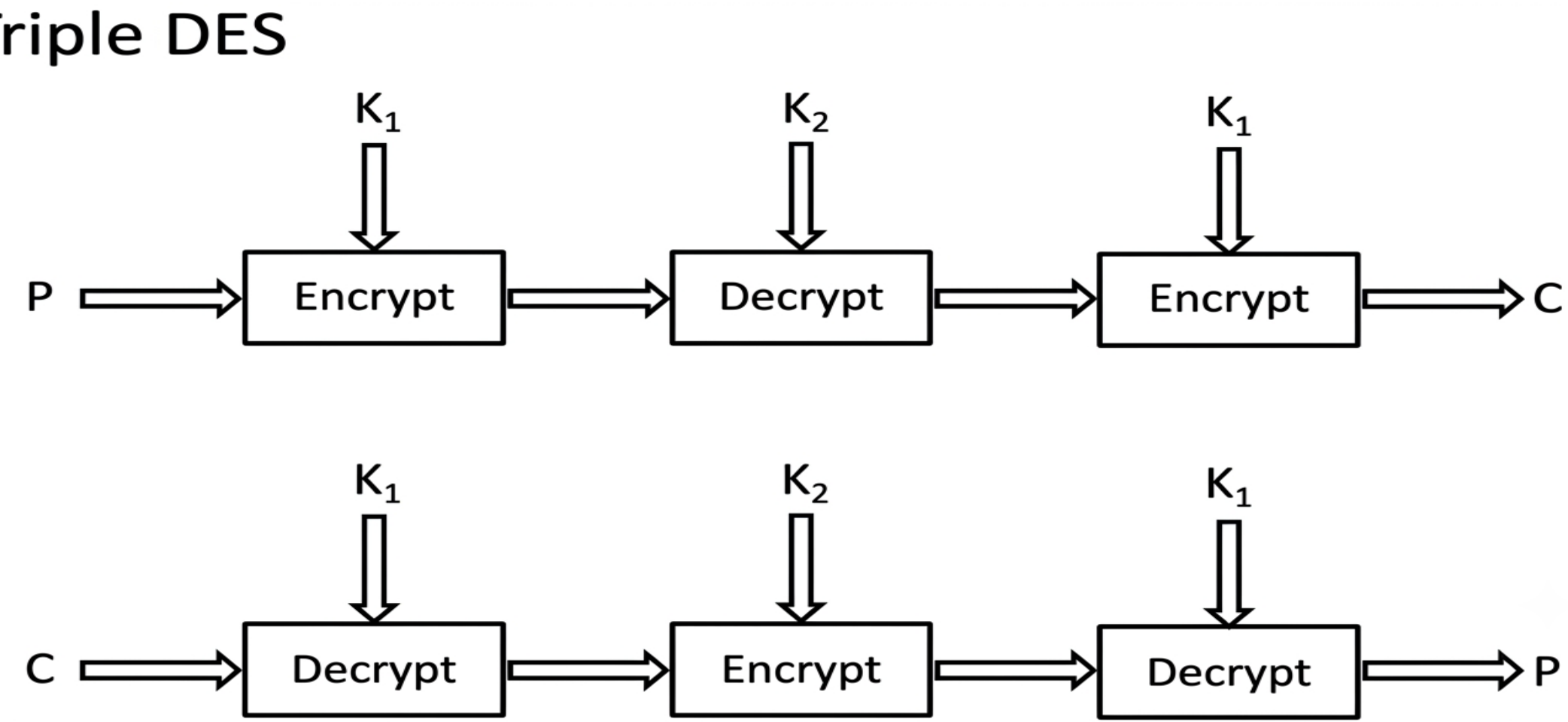


Figure 6.15 *Tables for meet-in-the-middle attack*



Triple DES (3DES)

- DES is applied **three times**.
- It uses an **Encrypt–Decrypt–Encrypt (EDE)** process.



Drawback of Multiple Encryption

- DES is performed **two or three times**.
- More processing is required.
- Therefore, it is **slower than single DES**.
- More keys must also be managed.

Double DES → 2 DES operations

Triple DES → 3 DES operations

More operations → More processing time

Security of DES

According to **Forouzan, Section 6.5**:

1. Brute-Force Attack

- DES key = **56 bits**
- Attacker tries possible keys.
- Key space = 2^{56}
- DES has **key-complement weakness**.
- About 2^{55}encryptions can break DES.

meaning:

Try keys until the correct key is found.

2. Differential Cryptanalysis

- Studies **input and output differences**.
- Uses **chosen plaintexts**.
- DES was designed to resist this attack.
- Needs about 2^{47}chosen plaintexts.
- Such a large number is impractical.

meaning:

Study how input differences change the output.

3. Linear Cryptanalysis

- Studies **linear relationships between bits**.
- Uses **known plaintexts**.
- DES is more vulnerable to this than differential cryptanalysis.
- Needs about 2^{43}known plaintext pairs.
- Still difficult in practice.

meaning:

Find simple relationships between plaintext, ciphertext, and key bits.

Remember

Security of DES = Brute Force + Differential + Linear